
Design Document

Team 09

2252709 Xuanhe Yang

2251548 Jingxiao Han

2251653 Tong Li

2253715 Fubin Chen

Abstract

This design document outlines the Smart Maintenance Platform for Aero Engine Industrial Equipment (Proactive Shield), a comprehensive solution that leverages Skip-GANomaly and CNN-LSTM deep learning models to enable predictive maintenance for aerospace engine components. The system architecture employs a multi-layered approach separating user interaction, security, business logic, and data persistence concerns, with Vue.js frontend, Spring Boot backend services, and a hybrid database strategy optimized for both transactional and time-series data. Key features include real-time monitoring, anomaly detection with $> 95\%$ accuracy, remaining useful life prediction, alert management, and reporting capabilities, collectively targeting 25-30% reduction in unplanned downtime, 20-25% decrease in maintenance costs, and 15% improvement in overall equipment effectiveness, all while meeting strict performance requirements and aviation industry standards.

Team Introduction

This project was collaboratively completed by the following team members:

Xuanhe Yang

Student ID: 2252709

Phone: +86 18937082731

Email: 2252709@tongji.edu.cn

Jingxiao Han

Student ID: 2251548

Phone: +86 13817024314

Email: 2251548@tongji.edu.cn

Tong Li

Student ID: 2251653

Phone: +86 18545556320

Email: 2251653@tongji.edu.cn

Fubin Chen

Student ID: 2253715

Phone: +86 15159581396

Email: 1958440015@qq.com

Table of Contents

Chapter1	Document Information	5
1	Document Control Information	5
2	Revision History	5
3	Approvals	5
4	Document Purpose	5
5	Audience	5
6	References	6
Chapter2	Introduction	7
1	Project Overview	7
2	System Purpose	7
2.2.1	Primary Objectives	7
2.2.2	Secondary Benefits	7
3	System Scope	8
2.3.1	In-Scope Components	8
2.3.2	Out-of-Scope Limitations	8
4	Design Goals and Constraints	9
2.4.1	Technical Design Goals	9
2.4.2	Operational Constraints	9
5	Assumptions and Dependencies	9
2.5.1	Data Assumptions	9
2.5.2	Technical Dependencies	10
2.5.3	Organizational Dependencies	10
Chapter3	Requirements and Work Breakdown Structure Alignment	11
1	Overview	11
2	Requirements Breakdown Structure (RBS)	11
3.2.1	Report Generation Module	11
3.2.2	Device Management Module	11
3.2.3	User Management Module	11
3.2.4	Monitor Center Module	12
3.2.5	Alert System Module	12
3.2.6	Forecast Service Module	12
3.2.7	Model Management Module	12
3	Work Breakdown Structure (WBS)	12
3.3.1	Initiation Phase (WBS 1)	12
3.3.2	Planning Phase (WBS 2)	13
3.3.3	Design Phase (WBS 3)	13
3.3.4	Development Phase (WBS 4)	13
3.3.5	Testing Phase (WBS 5)	14
3.3.6	Deployment and Launch Phase (WBS 6)	14
3.3.7	Closure Phase (WBS 7)	14
4	RBS-WBS-Module Alignment Matrix	14

5	Traceability and Coverage Analysis	14
3.5.1	Requirements Coverage	14
3.5.2	Work Package Efficiency	15
3.5.3	Module Integration	15
6	Implementation Validation	15
3.6.1	Stakeholder Review	15
3.6.2	Development Tracking	16
3.6.3	Testing Alignment	16
Chapter4 System Architecture		17
1	Architecture Overview	17
2	Detailed Architecture Layers	17
4.2.1	User Interaction Layer	17
4.2.2	Security Layer	18
4.2.3	Application Logic Layer	18
4.2.4	Application Data Layer	19
4.2.5	Data Source	19
3	Architectural Benefits	20
4	Component Interactions and Data Flow	20
5	Deployment Architecture	21
Chapter5 Class Design		22
1	Overview	22
2	Design Patterns	22
3	Core Domain Models	22
5.3.1	Alert Entity	22
5.3.2	Component Entity	23
4	Subsystem Class Designs	24
5.4.1	Alert System	24
5.4.2	Component System	26
5.4.3	Data System	28
5.4.4	Model System	32
5.4.5	Report System	34
5.4.6	Simulation System	37
5.4.7	User System	39
5	Class Relationships	42
Chapter6 Data Design		43
1	Data Models	43
2	Entity Relationships	45
3	Database Architecture Design	46
4	Data Access Layer	46
Chapter7 Module Designs		48
1	Device Center Module	48
7.1.1	Purpose and Functionality	48
7.1.2	Module Components	48
7.1.3	Data Flow	49
2	Monitoring Center Module	49
7.2.1	Purpose and Functionality	49

7.2.2	Module Components	50
7.2.3	Data Flow	50
3	Simulation and Prediction Module	51
7.3.1	Purpose and Functionality	51
7.3.2	Module Components	51
7.3.3	Data Flow	52
4	Alert System Module	52
7.4.1	Purpose and Functionality	52
7.4.2	Module Components	52
7.4.3	Data Flow	53
5	Reporting System Module	53
7.5.1	Purpose and Functionality	53
7.5.2	Module Components	53
7.5.3	Data Flow	54
6	User Management Module	54
7.6.1	Purpose and Functionality	54
7.6.2	Module Components	55
7.6.3	Data Flow	55
Chapter8	Machine Learning Model Design	56
1	Model Selection Rationale	56
8.1.1	For Anomaly Detection	56
8.1.2	For Remaining Useful Life Prediction	56
2	Skip-GANomaly Model Architecture	56
8.2.1	Model Structure	57
8.2.2	Training Approach	57
8.2.3	Anomaly Detection Process	58
3	CNN-LSTM Model Architecture	58
8.3.1	Model Structure	58
8.3.2	Training Approach	59
8.3.3	Model Evaluation and Performance Targets	60
4	Model Integration and Deployment Strategy	60
8.4.1	Model Selection Criteria	60
8.4.2	Implementation Considerations	60
Chapter9	User Interface Design	61
1	Design Principles	61
2	UI Components	61
9.2.1	Dashboard	61
9.2.2	Device Detail View	61
9.2.3	Monitoring Views	62
9.2.4	Alert Management	62
9.2.5	Reporting Interface	62
3	Navigation Structure	62
4	Responsive Design Strategy	63

Chapter10	Interface Specifications	64
1	API Design	64
10.1.1	API Principles	64
10.1.2	Authentication and Authorization	64
10.1.3	Major Endpoints	64
2	External Interfaces	66
10.2.1	Data Import and Export	66
10.2.2	Device Integration	66
10.2.3	Alert and Notification System	66
3	User Interface to Backend Integration	66
Chapter11	Non-Functional Design Aspects	68
1	Performance Design	68
11.1.1	Response Time Goals	68
11.1.2	Throughput Requirements	68
11.1.3	Optimization Strategies	68
2	Security Design	69
11.2.1	Authentication and Authorization	69
11.2.2	Data Protection	69
11.2.3	API Security	69
11.2.4	Security Monitoring	69
3	Scalability Design	69
11.3.1	Horizontal Scaling	70
11.3.2	Vertical Scaling	70
11.3.3	Elastic Capacity	70
4	Availability and Reliability Design	70
11.4.1	Availability Targets	70
11.4.2	Reliability Features	70
11.4.3	Monitoring and Recovery	70
Chapter12	Appendices	71
1	Glossary	71
2	Design Decisions Log	71
3	References	72

Chapter1 Document Information

1 Document Control Information

Document Title	Smart Maintenance Platform for Aero Engine Industrial Equipment
Version	1.2
Date	May 15, 2025
Status	END
Document Owner	Proactive Shield Team

Table 1.1: Document Control Information

2 Revision History

Version	Date	Description	Author
0.1	March 30, 2025	Initial structure	Fubin Chen
0.5	April 3, 2025	Architecture draft	Fubin Chen
1.0	April 6, 2025	Completed initial phase content	Project Team
1.1	April 20, 2025	Updated module design and architecture details	Project Team
1.2	May 15, 2025	Integrated development phase results and test outcomes	Project Team

Table 1.2: Revision History

3 Approvals

Name	Role	Approval Date	Signature
Xuanhe Yang	Project Manager	June 14th	YANGXUANHE
Jingxiao Han	Team Member	June 14th	HANJINGXIAO
Tong Li	Team Member	June 14th	LITONG
Fubin Chen	Team Member	June 14th	CHENFUBIN

Table 1.3: Approvals

4 Document Purpose

This document defines the system architecture, components, and design specifications for the Smart Maintenance Platform for Aero Engine Industrial Equipment. It serves as the primary reference for development teams and stakeholders throughout the project lifecycle, detailing design decisions, technical implementations, and functional modules.

5 Audience

- Development team members
- Project stakeholders

- Quality assurance team
- Operations team
- User training personnel

6 References

- Project Charter (March 29, 2025)
- Requirements Specification (April 3, 2025)
- Risk Management Plan (April 5, 2025)
- First Biweekly Report (April 6, 2025)
- Second Biweekly Report (April 20, 2025)
- Third Biweekly Report (May 11, 2025)
- System Test Report (May 10, 2025)

Chapter2 Introduction

1 Project Overview

The Smart Maintenance Platform for Aero Engine Industrial Equipment (Proactive Shield) represents a revolutionary approach to industrial maintenance, leveraging cutting-edge artificial intelligence and deep learning technologies to transform traditional reactive maintenance strategies into proactive, predictive solutions. This comprehensive platform integrates advanced neural network models, including Skip-GANomaly and CNN-LSTM architectures, with real-time sensor data analysis to provide unprecedented insights into equipment health and performance.

The system addresses critical challenges in the aviation industry where unplanned equipment failures can result in significant operational disruptions, safety concerns, and substantial financial losses. By analyzing multi-dimensional sensor data including temperature, vibration, pressure, and other operational parameters, the platform enables maintenance teams to transition from time-based or failure-driven maintenance schedules to intelligent, condition-based maintenance strategies.

Built as a cloud-native web application, Proactive Shield provides a centralized monitoring and analysis platform that supports distributed equipment management across multiple facilities. The system's modular architecture ensures scalability and adaptability to various operational environments while maintaining high performance and reliability standards essential for mission-critical applications.

2 System Purpose

The Smart Maintenance Platform is designed to achieve multiple strategic objectives that directly impact operational efficiency, cost management, and safety standards:

2.2.1 Primary Objectives

- **Anomaly Detection:** Implement advanced deep learning algorithms (Skip-GANomaly, CNN-LSTM) to achieve > 95% accuracy in detecting equipment anomalies and potential failure patterns before they manifest as operational issues
- **Predictive Analytics:** Accurately predict remaining useful life (RUL) of critical aero engine components through sophisticated analysis of historical maintenance records, operational data, and real-time sensor inputs
- **Operational Efficiency:** Reduce unplanned downtime by 25–30% through proactive maintenance scheduling, enabling continuous operation and minimizing production disruptions
- **Resource Optimization:** Improve Overall Equipment Effectiveness (OEE) by 15 percentage points through optimized maintenance scheduling, better resource allocation, and enhanced equipment utilization
- **Decision Support:** Provide comprehensive monitoring, reporting, and analytics capabilities that support data-driven decision making across all levels of maintenance operations
- **Cost Reduction:** Achieve 20–25% reduction in annual maintenance costs through optimized maintenance schedules, reduced emergency repairs, and extended equipment lifecycles

2.2.2 Secondary Benefits

- Enhanced safety through early detection of potential failure modes

- Improved maintenance team productivity and expertise development
- Better inventory management through predictive spare parts planning
- Compliance support for regulatory maintenance requirements
- Knowledge preservation and sharing across maintenance teams

3 System Scope

2.3.1 In-Scope Components

Cloud-based Predictive Maintenance Platform Development

- **Centralized Web Monitoring Platform:** Development of a browser-based unified monitoring portal supporting centralized management of multi-site equipment with responsive design for cross-device compatibility, integrated access control, and single sign-on functionality
- **Real-time Data Visualization:** Implementation of dynamic dashboards with millisecond-level refresh capabilities for equipment parameters (vibration, temperature, pressure, etc.), offering multiple visualization modes including trend charts, heat maps, and statistical displays with customizable monitoring thresholds
- **AI Analytics Engine:** Deployment of LSTM neural networks and Skip-GANomaly models for failure pattern recognition and remaining useful life prediction, supporting online model updates with automated evaluation targeting $> 95\%$ accuracy

System Integration Capabilities

- **IoT Device Interface:** Development of standardized device communication protocol adaptation layer supporting major industrial protocols (OPC-UA, Modbus, MQTT), ensuring compatibility with $> 90\%$ of existing sensor equipment in industrial environments
- **Data Collection and Transmission:** Establishment of reliable data pipelines from industrial equipment to analytics platform, handling multi-protocol connectivity, performing edge preprocessing (filtering, compression), and ensuring secure, fault-tolerant transmission with buffering and auto-recovery capabilities

Core Functional Modules

- **Real-time Monitoring Dashboard:** Comprehensive equipment operation status display including device detail cards, overall health trend charts, real-time energy consumption curves, and real-time parameter monitoring
- **Intelligent Alert System:** Flexible notification system featuring alarm device information, alert timestamps, severity levels, detailed alert descriptions, and statistical monitoring data with weekly reporting capabilities
- **Predictive Maintenance Tools:** Advanced analysis capabilities enabling model and equipment selection for simulation, generating equipment health indices and remaining lifespan predictions with accuracy tracking

2.3.2 Out-of-Scope Limitations

- **Hardware Procurement:** No new sensor hardware or monitoring equipment acquisition
- **Physical Modifications:** Equipment retrofitting or physical infrastructure changes
- **Enterprise Integration:** Deep integration with existing ERP or legacy enterprise systems
- **Mobile Applications:** Development of standalone native mobile applications (web browser access supported only)
- **Custom Hardware:** Design or manufacturing of specialized monitoring hardware

4 Design Goals and Constraints

2.4.1 Technical Design Goals

- **Modular Architecture:** Create a modular and extensible system architecture that supports easy integration of new features, algorithms, and data sources while maintaining system stability and performance
- **High Availability:** Ensure system availability $\geq 99.5\%$ with robust fault tolerance, automatic failover capabilities, and comprehensive backup systems to meet mission-critical operational requirements
- **Real-time Performance:** Support real-time monitoring and analysis with response times ≤ 3 seconds for dashboard updates and $\leq 500\text{ms}$ for critical alert processing to enable immediate response to equipment issues
- **Prediction Accuracy:** Achieve and maintain high prediction accuracy ($\geq 90\%$) for remaining useful life estimates and $\geq 95\%$ accuracy for anomaly detection through continuous model optimization and validation
- **User Experience:** Provide intuitive, responsive user interfaces that require minimal training while supporting complex analytical workflows and customizable views for different user roles
- **Integration Capability:** Enable seamless integration with existing maintenance management systems, SCADA platforms, and enterprise databases through standardized APIs and protocols

2.4.2 Operational Constraints

- **Infrastructure Compatibility:** Must operate within existing IT infrastructure without requiring major hardware upgrades or network modifications
- **Security Compliance:** Must comply with aviation industry data security standards including data encryption, access controls, and audit trail requirements
- **Data Processing:** Must efficiently handle high-volume sensor data streams (potentially thousands of data points per second) while maintaining real-time processing capabilities
- **Sensor Compatibility:** Must support various sensor types and data formats commonly used in aero engine monitoring without requiring sensor replacement or modification
- **Bandwidth Limitations:** Must operate effectively with limited bandwidth in some deployment scenarios through efficient data compression and intelligent data prioritization
- **Regulatory Requirements:** Must support compliance with aviation maintenance regulations and provide necessary documentation and audit trails for regulatory inspections

5 Assumptions and Dependencies

2.5.1 Data Assumptions

- Availability of comprehensive historical engine operational data spanning multiple years for effective model training and validation
- Access to real-time sensor data with sufficient granularity and frequency to support predictive analytics
- Data quality meets minimum standards for machine learning applications with acceptable levels of completeness and accuracy
- Historical maintenance records are available in structured or semi-structured formats suitable for analysis

2.5.2 Technical Dependencies

- Reliable network connectivity between sensor systems and the central platform for continuous data transmission
- Adequate computational resources for deep learning model training and real-time inference processing
- Integration capabilities with existing maintenance management systems through available APIs or data export mechanisms
- Cloud infrastructure availability with appropriate security controls and compliance certifications

2.5.3 Organizational Dependencies

- Stakeholder commitment to data-driven maintenance approaches and willingness to adapt existing workflows
- Technical team availability with appropriate expertise in machine learning, web development, and system integration
- Management support for the digital transformation initiative including necessary resource allocation and change management
- Collaboration from maintenance teams for system validation, testing, and feedback during development and deployment phases

Chapter3 Requirements and Work Breakdown Structure Alignment

1 Overview

This chapter establishes the clear relationship between the Requirements Breakdown Structure (RBS), Work Breakdown Structure (WBS), and the system's design modules. The alignment ensures that all functional requirements are systematically addressed through specific work packages and implemented in corresponding system modules.

2 Requirements Breakdown Structure (RBS)

The Requirements Breakdown Structure organizes the system requirements in a hierarchical tree structure, providing a comprehensive view of all functional capabilities that the Smart Maintenance Platform must deliver.

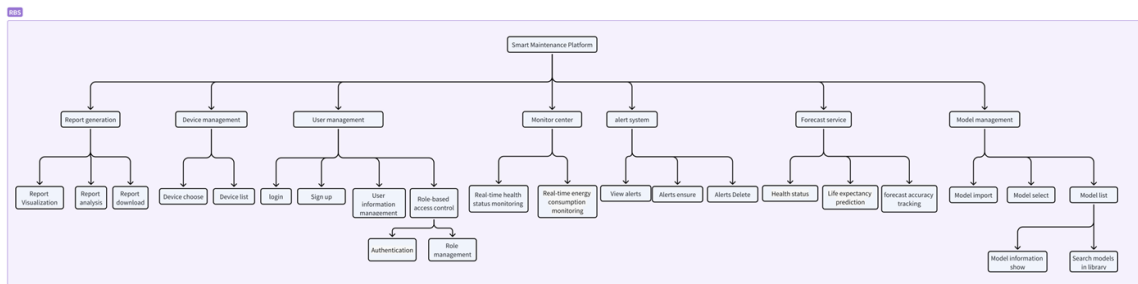


Figure 3.1: Requirements Breakdown Structure

As illustrated in Figure 3.1, the RBS is organized into seven main functional modules:

3.2.1 Report Generation Module

- **Report Visualization:** Interactive charts and graphs for data presentation
- **Report Analysis:** Statistical analysis and trend identification
- **Report Download:** Export capabilities in multiple formats (PDF, Excel, CSV)

3.2.2 Device Management Module

- **Device Choose:** Selection interface for component analysis
- **Device List:** Comprehensive inventory of monitored components

3.2.3 User Management Module

- **Login:** Secure user authentication
- **Sign Up:** New user registration process
- **User Information Management:** Profile and account management
 - Authentication: Credential verification

– Role Management: Permission-based access control

- **Role-based Access Control:** Hierarchical permission system

3.2.4 Monitor Center Module

- **Real-time Health Status Monitoring:** Continuous component health tracking
- **Real-time Energy Consumption Monitoring:** Power usage analysis and optimization

3.2.5 Alert System Module

- **View Alerts:** Alert dashboard and notification center
- **Alerts Ensure:** Alert confirmation and acknowledgment
- **Alerts Delete:** Alert lifecycle management

3.2.6 Forecast Service Module

- **Health Status Prediction:** Component condition forecasting
- **Life Expectancy Prediction:** Remaining useful life estimation
- **Forecast Accuracy Tracking:** Model performance monitoring

3.2.7 Model Management Module

- **Model Import:** AI model integration capabilities
- **Model Select:** Model selection for analysis tasks
- **Model List:** Available model inventory
- **Model Information Show:** Model metadata and specifications
- **Search Models in Library:** Model discovery and filtering

3 Work Breakdown Structure (WBS)

The Work Breakdown Structure decomposes the project work into manageable components, organizing development activities across the project lifecycle from initiation to closure.

As shown in Figure 3.2, the WBS is structured across seven major phases:

3.3.1 Initiation Phase (WBS 1)

- **1.1 Requirements Analysis and Confirmation:** Stakeholder requirement gathering
- **1.2 Project Charter Development:** Project scope and objectives definition
- **1.3 Project Scope Definition:** Boundary establishment and deliverable identification
- **1.4 Stakeholder Identification:** Key participant mapping and engagement planning
- **1.5 Resource Allocation:** Team assignment and resource planning

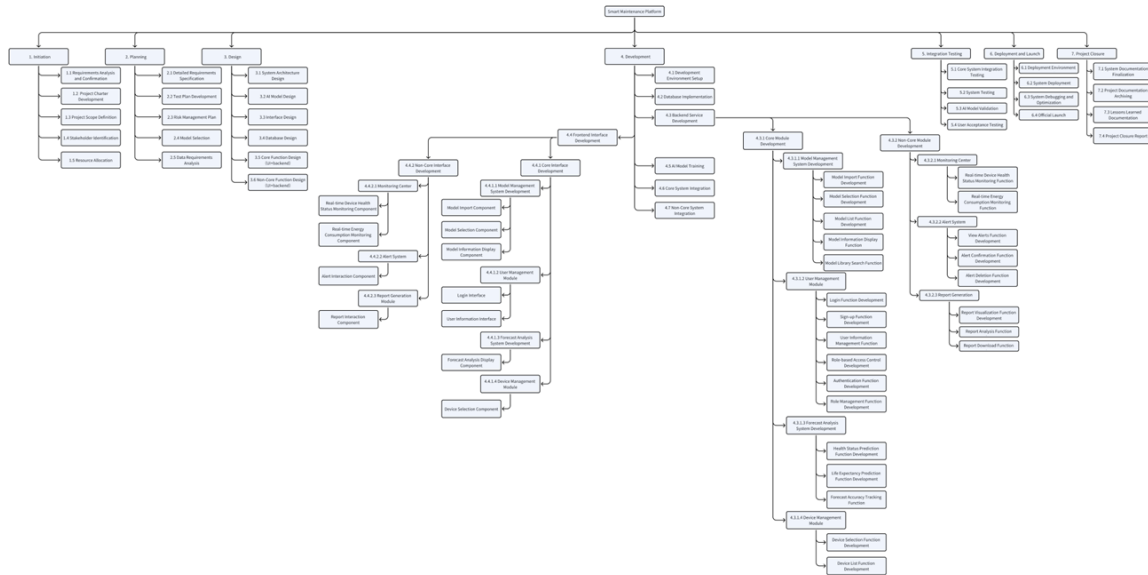


Figure 3.2: Work Breakdown Structure

3.3.2 Planning Phase (WBS 2)

- **2.1 Detailed Requirements Specification:** Technical requirement documentation
- **2.2 Test Plan Development:** Quality assurance strategy
- **2.3 Risk Management Plan:** Risk identification and mitigation strategies
- **2.4 Model Selection:** AI model evaluation and selection
- **2.5 Data Requirements Analysis:** Data architecture and flow design

3.3.3 Design Phase (WBS 3)

- **3.1 System Architecture Design:** Overall system structure and component relationships
- **3.2 AI Model Design:** Machine learning model architecture specification
- **3.3 Interface Design:** User interface and API design
- **3.4 Database Design:** Data model and storage architecture
- **3.5 Core Function Design (UI+Backend):** Primary feature implementation design
- **3.6 Non-Core Function Design (UI+Backend):** Secondary feature implementation design

3.3.4 Development Phase (WBS 4)

- **4.1 Development Environment Setup:** Infrastructure and tooling preparation
- **4.2 Database Implementation:** Data persistence layer development
- **4.3 Backend Service Development:** Business logic implementation
 - 4.3.1 Core Function Development with Unit System
 - 4.3.2 Non-Core Function Development with Unit Test
- **4.4 Frontend Interface Development:** User interface implementation
 - 4.4.1 Core Interface Development

– 4.4.2 Non-Core Interface Development

- **4.5 AI Model Training:** Machine learning model development and optimization
- **4.6 Core System Integration:** Primary component integration
- **4.7 Non-Core System Integration:** Secondary component integration

3.3.5 Testing Phase (WBS 5)

- **5.1 Core System Integration Testing:** Primary functionality validation
- **5.2 System Testing:** Comprehensive system validation
- **5.3 AI Model Validation:** Machine learning model performance verification
- **5.4 User Acceptance Testing:** Stakeholder validation and approval

3.3.6 Deployment and Launch Phase (WBS 6)

- **6.1 Deployment Environment:** Production infrastructure preparation
- **6.2 System Deploy:** Application deployment and configuration
- **6.3 System Debugging:** Issue resolution and optimization
- **6.4 Official Launch:** Production system activation

3.3.7 Closure Phase (WBS 7)

- **7.1 System Documentation Finalization:** Technical documentation completion
- **7.2 Project Documentation Archiving:** Knowledge management and storage
- **7.3 Lessons Learned Documentation:** Experience capture and sharing
- **7.4 Project Closure Report:** Final project assessment and handover

4 RBS-WBS-Module Alignment Matrix

Table 3.1 demonstrates the systematic relationship between requirements, work packages, and implementation modules:

5 Traceability and Coverage Analysis

3.5.1 Requirements Coverage

The alignment analysis confirms that:

- **100% Requirements Coverage:** Every RBS requirement maps to specific WBS tasks and design modules
- **Bidirectional Traceability:** Clear mapping from requirements through work packages to implementation
- **No Orphaned Work:** All WBS tasks contribute to fulfilling identified requirements
- **Comprehensive Module Design:** Each design module addresses complete functional domains

Table 3.1: RBS-WBS-Module Alignment Matrix

RBS Module	Primary WBS Tasks	Design Module	Implementation Components
Report Generation	3.3, 4.4.2, 5.2	Reporting System Module	ReportController, ReportService, Export utilities
Device Management	3.4, 4.2, 4.3.1	Device Center Module	ComponentController, ComponentService, ComponentRepository
User Management	3.5, 4.3.1, 4.4.1	User Management Module	UserController, UserService, JWT Authentication
Monitor Center	3.2, 4.5, 4.6	Monitoring Center Module	MonitorController, DataService, Real-time processors
Alert System	3.6, 4.3.2, 4.4.2	Alert System Module	AlertController, AlertService, Notification engine
Forecast Service	3.2, 4.5, 5.3	Simulation and Prediction Module	SimulationController, ML models, Prediction engine
Model Management	3.2, 4.5, 4.7	Model Management System	ModelController, ModelService, Model repository

3.5.2 Work Package Efficiency

The WBS structure ensures:

- **Logical Progression:** Work packages follow natural development dependencies
- **Balanced Distribution:** Work is evenly distributed across development phases
- **Risk Mitigation:** Critical path activities are identified and protected
- **Quality Assurance:** Testing activities are integrated throughout the lifecycle

3.5.3 Module Integration

The design modules provide:

- **Cohesive Functionality:** Each module addresses related requirements as a unit
- **Clear Interfaces:** Well-defined boundaries between modules support integration
- **Scalable Architecture:** Modular design enables independent scaling and enhancement
- **Maintainable Structure:** Clear separation of concerns simplifies maintenance

6 Implementation Validation

The alignment between RBS, WBS, and design modules has been validated through:

3.6.1 Stakeholder Review

- Requirements completeness confirmed by business stakeholders
- Technical feasibility validated by development team
- Integration points approved by system architects

3.6.2 Development Tracking

- WBS tasks mapped to project schedule milestones
- Progress tracking aligned with requirement completion
- Quality gates tied to specific RBS requirement validation

3.6.3 Testing Alignment

- Test cases derived directly from RBS requirements
- Integration tests validate module interactions
- Acceptance criteria mapped to stakeholder requirements

This systematic alignment ensures that the Smart Maintenance Platform development process maintains clear traceability from business requirements through implementation to final delivery, supporting both project management effectiveness and system quality assurance.

Chapter4 System Architecture

1 Architecture Overview

The Smart Maintenance Platform for Aero Engine Industrial Equipment employs a layered architecture with clear separation of concerns. As illustrated in the System Architecture Diagram, the architecture consists of five distinct layers:

1. **User Interaction Layer:** Handles all user interface rendering and client-side logic
2. **Security Layer:** Provides authentication, authorization, and security controls
3. **Application Logic Layer:** Contains the core business logic with controllers and services
4. **Application Data Layer:** Manages data models, DTOs, and repository interfaces
5. **Data Source:** Provides persistent storage for application data

This layered approach enables modularity, maintainability, and testability by enforcing clear boundaries between system components and facilitating independent development and evolution of each layer.

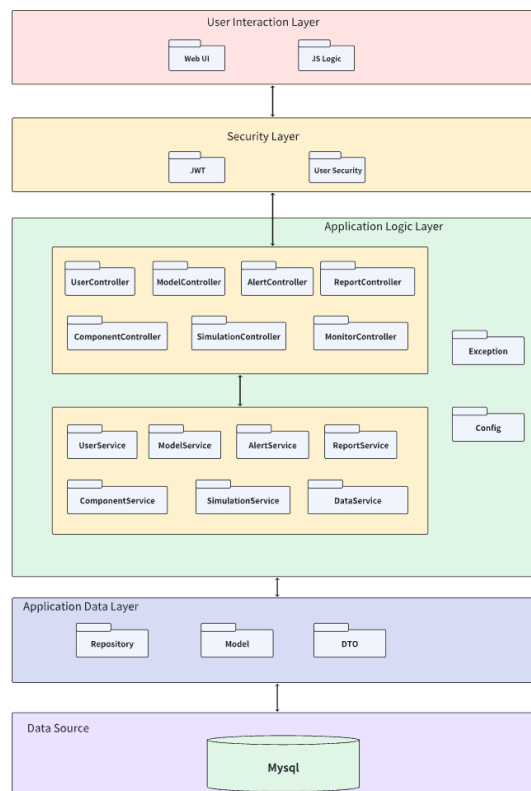


Figure 4.1: System Architecture Diagram

2 Detailed Architecture Layers

4.2.1 User Interaction Layer

The User Interaction Layer comprises two main components that work together to provide a comprehensive user experience:

- **Web UI:** Implements all visual components, including dashboards, forms, and data visualizations using responsive design principles to ensure optimal display across different devices and screen sizes
- **JS Logic:** Contains client-side business logic, state management, API integration code, and interactive functionality that operates independently from UI components

This layer is responsible for presenting information to users and capturing user interactions, providing an intuitive interface for system operations including device monitoring, alert management, and report generation.

4.2.2 Security Layer

The Security Layer serves as a critical gateway between the User Interaction Layer and the Application Logic Layer, ensuring system security through two main components:

- **JWT (JSON Web Token):** Implements stateless authentication and authorization mechanism, providing secure token-based access control for API endpoints
- **User Security:** Comprehensive security services including user authentication, session management, input validation, CSRF protection, and access control enforcement

This layer ensures that all requests to the application are properly authenticated and authorized before reaching the application logic components, maintaining system integrity and protecting sensitive data.

4.2.3 Application Logic Layer

The Application Logic Layer forms the core of the system, containing the business logic implemented through controllers and services organized in a clear hierarchical structure.

Controller Layer

The controller layer exposes RESTful API endpoints and handles HTTP request processing:

- **UserController:** Manages user-related operations including authentication, registration, profile management, and user information retrieval
- **ModelController:** Handles AI model management operations such as model listing, selection, and metadata management
- **AlertController:** Processes alert generation, notification management, confirmation, and alert status updates
- **ReportController:** Manages report generation, statistical analysis, and data export operations
- **ComponentController:** Controls aero engine component operations, device management, and component status monitoring
- **SimulationController:** Handles simulation requests, model execution, and result processing for predictive analysis
- **MonitorController:** Manages real-time monitoring operations, health data collection, and energy consumption tracking

The layer also includes supporting components:

- **Exception:** Centralized exception handling for consistent error responses
- **Config:** Application configuration management for system settings

Service Layer

The service layer implements the core business logic and rules:

- **UserService:** Implements user management business logic, authentication workflows, and user data validation
- **ModelService:** Implements model management functionality, including model validation and metadata processing
- **AlertService:** Implements alert generation algorithms, notification logic, and alert lifecycle management
- **ReportService:** Implements report generation engines, data aggregation, and statistical calculations
- **ComponentService:** Implements component management business rules, device registration, and status evaluation
- **SimulationService:** Implements simulation algorithms, model inference, and predictive analytics
- **DataService:** Implements data processing, transformation logic, and health index calculations

Each service component corresponds to its respective controller, creating a consistent and maintainable architectural pattern throughout the application.

4.2.4 Application Data Layer

The Application Data Layer provides comprehensive data access and manipulation capabilities through three main component types:

- **Repository:** Data access interfaces that define CRUD operations and complex queries for each entity, abstracting the underlying data persistence mechanisms
- **Model:** Domain entities that represent the core business objects including User, Component, Alert, Model, and Data entities with their relationships and constraints
- **DTO (Data Transfer Objects):** Lightweight objects used for data exchange between layers, providing controlled data exposure and transformation capabilities

This layer ensures data consistency, implements business rules at the data level, and provides a clean abstraction for data operations across the application.

4.2.5 Data Source

The Data Source layer provides reliable and persistent storage for all application data:

- **MySQL:** Primary relational database system used for storing structured data including:
 - User information and authentication data
 - Component details and specifications
 - AI model metadata and configurations
 - Alert records and notification history
 - Sensor data and operational metrics
 - Report templates and generated reports
 - System configuration and audit logs

The database is designed with proper normalization, indexing, and constraint enforcement to ensure data integrity, performance, and scalability for industrial-scale operations.

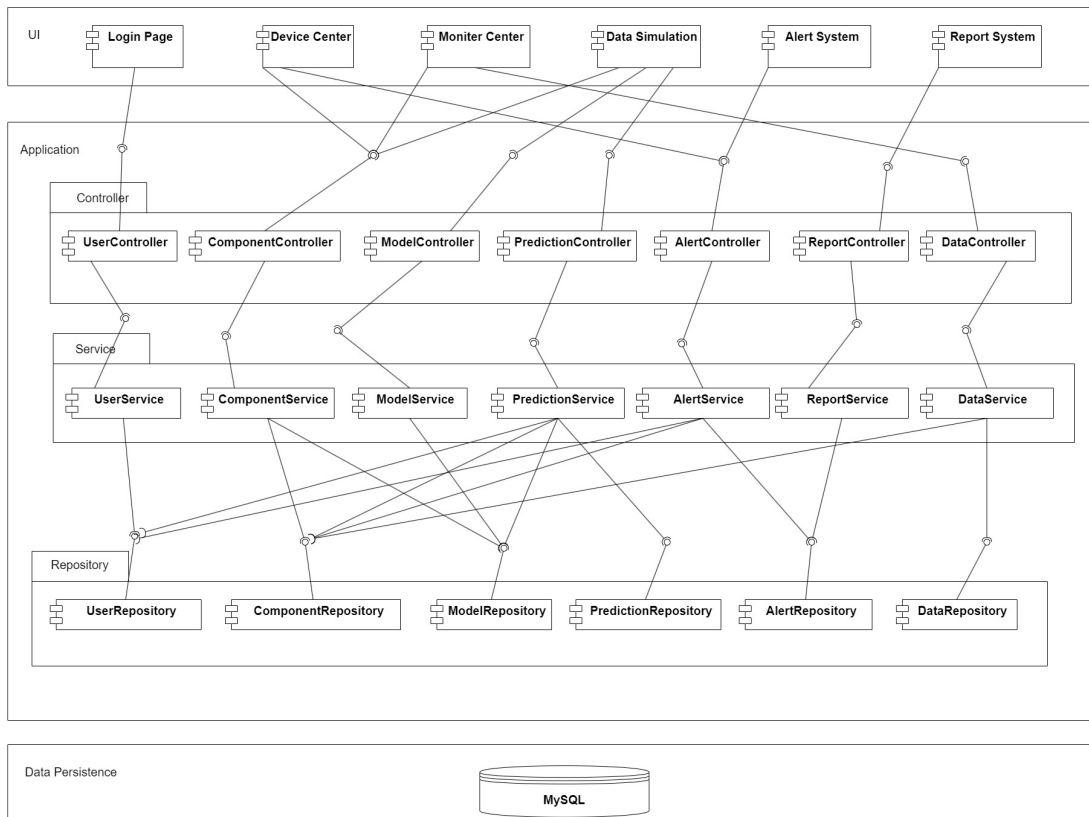
3 Architectural Benefits

This layered architecture provides several key advantages:

- **Separation of Concerns:** Each layer has clearly defined responsibilities, reducing complexity and improving maintainability
- **Scalability:** Individual layers can be scaled independently based on system load and performance requirements
- **Testability:** Each layer can be tested in isolation, enabling comprehensive unit and integration testing
- **Security:** Centralized security layer ensures consistent protection across all system components
- **Flexibility:** New features can be added or existing ones modified with minimal impact on other layers
- **Reusability:** Service components can be reused across different controllers and applications

4 Component Interactions and Data Flow

The Interface Diagram illustrates the comprehensive interactions between system components across all layers:



Interface Diagram

Figure 4.2: Interface Diagram

Key interaction patterns include:

1. **UI to Controller Flow:** Each UI module connects to one or more controllers that provide its required functionality. For example, the Device Center UI connects to ComponentController for device management operations.
2. **Controller to Service Flow:** Controllers delegate business logic to corresponding service components. Each controller has a direct relationship with its matching service (e.g., UserController uses UserService).

3. **Service to Repository Flow:** Services access data through repository interfaces. The diagram shows more complex relationships at this level, with services often using multiple repositories to fulfill their business requirements. For example, `PredictionService` connects to `PredictionRepository`, `ComponentRepository`, and `ModelRepository`.
4. **Repository to Database Flow:** All repositories connect to the MySQL database for persistent storage operations.

This flow demonstrates the system's adherence to clean architecture principles, with dependencies flowing inward from UI through controllers and services to repositories and the database.

5 Deployment Architecture

Based on the architectural design, the system will be deployed using a multi-tiered approach:

1. Client Tier:

- Web application served through standard web servers
- Static assets delivered via CDN for performance

2. API Tier:

- RESTful API endpoints implemented by controllers
- Load balanced across multiple instances for high availability
- JWT authentication enforcement at API gateway

3. Service Tier:

- Business logic services with Spring Boot containers
- Microservices architecture allowing independent scaling
- Docker containerization for consistent deployment

4. Data Tier:

- MySQL database with master-slave replication
- Data backup and recovery mechanisms
- Optional caching layer for performance optimization

The deployment architecture supports horizontal scaling at each tier, allowing the system to handle increasing loads as the number of monitored aero engine components grows.

Chapter5 Class Design

1 Overview

This chapter details the class design of the Smart Maintenance Platform for Aero Engine Industrial Equipment. The classes are organized according to the system's architectural layers and modules. Each class is defined with its attributes, methods, relationships, and responsibilities, providing a comprehensive view of the system's object-oriented design.

2 Design Patterns

The system implements several design patterns to promote maintainability, extensibility, and code reuse:

- **Model-View-Controller (MVC):** Separates application logic from presentation concerns
- **Repository Pattern:** Abstracts data access operations
- **Builder Pattern:** Used for complex object construction (e.g., DTOs)
- **Service Layer Pattern:** Encapsulates business logic in service components
- **Data Transfer Object (DTO):** Transfers data between layers
- **Interface Segregation:** Defines cohesive interfaces for specific functionality

3 Core Domain Models

5.3.1 Alert Entity

Attributes	
status: Status	Current status of the alert
confirmed: Boolean	Whether the alert has been confirmed
confidently: User	User who confirmed the alert
forecast: Forecast	Associated forecast data
id: Integer	Unique identifier for the alert
confirmedTime: LocalDateTime	Time when alert was confirmed
reserveTime: long	Reserved time period
alertDescription: String	Description of the alert
component: Component	Component that triggered the alert
alertTime: LocalDateTime	Time when alert was generated
Methods	
alertTime(): LocalDateTime	Getter for alert time
getId(): Integer	Getter for alert ID
getComponent(): Component	Getter for component
getStatus(): Status	Getter for alert status
getAlertDescription(): String	Getter for alert description
setDeviceId(String): void	Setter for device ID

setConfirmedBy(User): void	Setter for confirming user
setStatus(Status): void	Setter for alert status
setConfirmedTime(LocalDateTime): void	Setter for confirmation time
getComponentName(): void	Getter for component name
getConfirmedTime(): LocalDateTime	Getter for confirmation time
setComponent(Component): void	Setter for component
getUserTime(LocalDateTime): void	Setter for user time
getForecast(): Forecast	Getter for forecast
setConfirmedScopeIn(): void	Setter for confirmation scope
setId(Integer): void	Setter for alert ID

5.3.2 Component Entity

Attributes	
dataList: List<Data>	List of sensor data for the component
id: Integer	Unique identifier for the component
pic: String	Image path for the component
warningTime: LocalDateTime	Time of last warning
name: String	Name of the component
alerts: List<Alert>	List of alerts for this component
forecasts: List<Forecast>	List of forecasts for this component
user: User	User who owns this component
status: Integer	Current status of the component
Methods	
getAlerts(): List<Alert>	Retrieves all alerts for this component
getId(): Integer	Getter for component ID
setForecasts(List<Forecast>): void	Setter for forecasts list
setPic(String): void	Setter for component image
setAlerts(List<Alert>): void	Setter for alerts list
getName(): String	Getter for component name
getStatus(): Integer	Getter for component status
getPic(): String	Getter for component image
setWarningTime(LocalDateTime): void	Setter for warning time
getWarningTime(): LocalDateTime	Getter for warning time
setStatus(Integer): void	Setter for component status
getDataList(): List<Data>	Getter for sensor data list
getUser(): User	Getter for component owner
getForecasts(): List<Forecast>	Getter for forecasts list
setUser(User): void	Setter for component owner
setDataList(List<Data>): void	Setter for sensor data list
setId(Integer): void	Setter for component ID
setName(String): void	Setter for component name
getHealthIndex(): Double	Getter for component health index

getUnconfirmedAlerts(Map<String, Object>): ResponseEntity<Map<String, Object>>	Gets unconfirmed alerts
confirmAlerts(Map<String, Object>): ResponseEntity<Map<String, Object>>	Confirms alerts
getAlertsSummary(): ResponseEntity<Map<String, Object>>	Gets alert statistics

AlertService Interface

Attributes	
alertService: AlertService	Alert service implementation
dataRepository: DataRepository	Data access repository
alertRepository: AlertRepository	Alert data repository
userRepository: UserRepository	User data repository
Methods	
getUnconfirmedAlerts(Integer, String, String, String, int, Page<Alert>): int	Gets unconfirmed alerts
confirmAlerts(List<Integer>): int	Confirms multiple alerts
exportAlertsToXLSX(Integer): Resource	Exports alerts to Excel
getAlertsSummary(Integer): ResponseEntity<Map<String, Object>>	Gets alert summary
deleteAlerts(List<Integer>): int	Deletes multiple alerts
getAlerts(Integer, Map<String, Object>): ResponseEntity<Map<String, Object>>	Retrieves alerts

AlertRepository Interface

Methods	
confirmAlerts(List<Integer>, Integer, LocalDateTime, int): int	Confirms alerts by IDs
findUnconfirmedAlerts(Integer, String, LocalDateTime, LocalDateTime, Page<Alert>): Page<Alert>	Finds unconfirmed alerts
getAlertsSummary(Integer): List<Object[]>	Gets alert statistics
deleteByIds(List<Integer>): int	Deletes alerts by IDs
getAlerts(Integer, LocalDateTime, LocalDateTime, List<Object[]>): List<Object[]>	Gets alerts with criteria

5.4.2 Component System

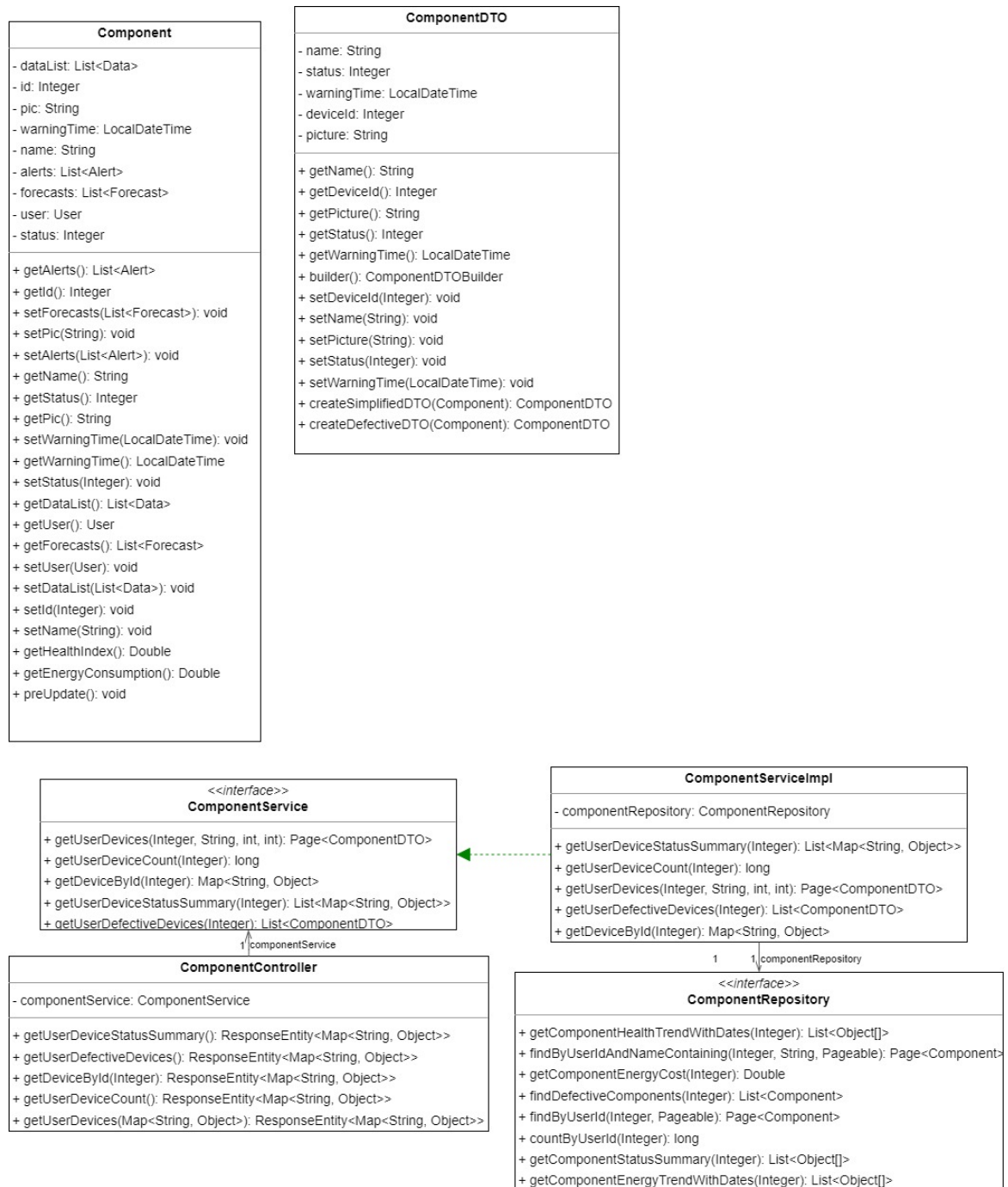


Figure 5.2: Component System Class Diagram

ComponentController Class

Attributes	
componentService: ComponentService	Service for component operations
Methods	
getUserDeviceStatusSummary(): ResponseEntity<Map<String, Object>>	Gets device status summary

getUserDefectiveDevices(): ResponseEntity<Map<String, Object>>	Gets defective devices
getDeviceById(Integer): ResponseEntity<Map<String, Object>>	Gets device by ID
getUserDeviceCount(): ResponseEntity<Map<String, Object>>	Gets device count
getUserDevices(Map<String, Object>): ResponseEntity<Map<String, Object>>	Gets user devices

ComponentService Interface

Methods	
getUserDeviceStatusSummary(Integer) List<Map<String, Object>>	Gets device status summary
getUserDeviceCount(Integer): long	Gets user device count
getUserDevices(Integer, String, int, int): Page<ComponentDTO>	Gets user devices with pagination
getUserDefectiveDevices(Integer): List<ComponentDTO>	Gets defective devices
getDeviceById(Integer): Map<String, Object>	Gets device by ID

ComponentServiceImpl Class

Attributes	
componentRepository: ComponentRepository	Component data repository
Methods	
getUserDeviceStatusSummary(Integer) List<Map<String, Object>>	Implementation of status summary
findByUserIdAndNameContaining(Integer, String, Pageable): Page<Component>	Finds components by user and name
getComponentEnergyCost(Integer): Double	Gets energy cost for component
findDefectiveComponents(Integer): List<Component>	Finds defective components
findByUserId(Integer, Pageable): Page<Component>	Finds components by user
countByUserId(Integer): long	Counts components by user
getComponentStatusSummary(Integer): List<Object[]>	Gets component status summary
getComponentEnergyTrendWithDates(Integer): List<Object[]>	Gets energy trend data

5.4.3 Data System

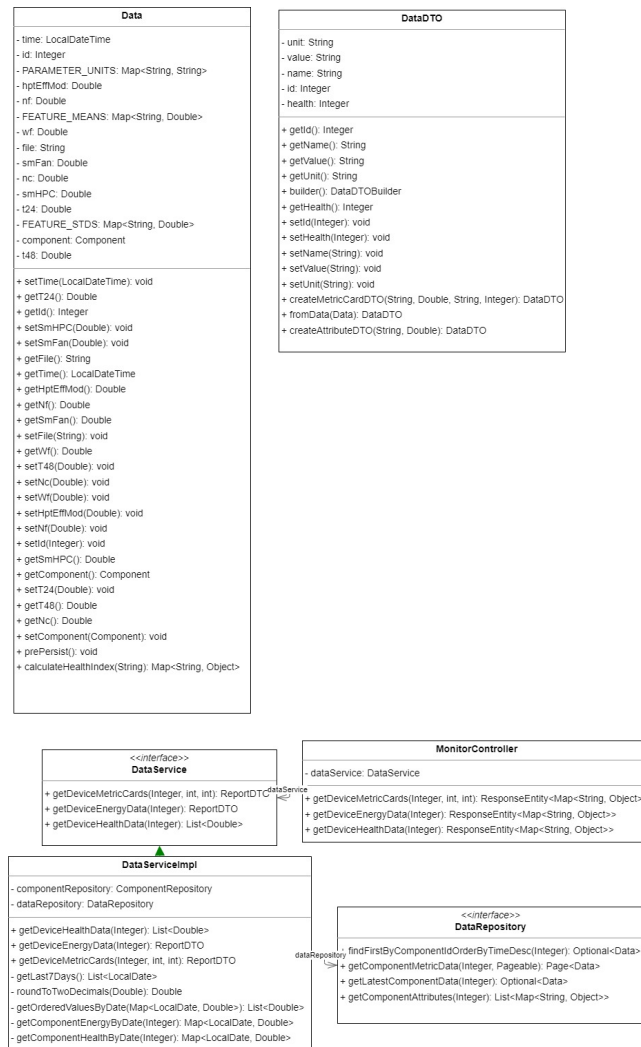


Figure 5.3: Data System Class Diagram

Data Entity

Attributes	
time: LocalDateTime	Timestamp of data collection
id: Integer	Unique identifier
PARAMETER_UNITS: Map<String, String>	Parameter units mapping
hptEffMod: Double	High pressure turbine efficiency
nf: Double	Fan speed
FEATURE_MEANS: Map<String, Double>	Feature means for normalization
wf: Double	Fuel flow
file: String	Data file path
smFan: Double	Fan stall margin
nc: Double	Core speed
smHPC: Double	HPC stall margin
t24: Double	Temperature at station 24

FEATURE_STDS: Map<String, Double>	Feature standard deviations
component: Component	Associated component
t48: Double	Temperature at station 48
Methods	
setTime(LocalDateTime): void	Sets data timestamp
getT24(): Double	Gets T24 temperature
getId(): Integer	Gets data ID
setSmHPC(Double): void	Sets HPC stall margin
setSmFan(Double): void	Sets fan stall margin
getFile(): String	Gets file path
getTime(): LocalDateTime	Gets timestamp
getHptEffMod(): Double	Gets HPT efficiency
getNf(): Double	Gets fan speed
getSmFan(): Double	Gets fan stall margin
setFile(String): void	Sets file path
getWf(): Double	Gets fuel flow
setT48(Double): void	Sets T48 temperature
setNc(Double): void	Sets core speed
setWf(Double): void	Sets fuel flow
setHptEffMod(Double): void	Sets HPT efficiency
setNf(Double): void	Sets fan speed
setId(Integer): void	Sets data ID
getSmHPC(): Double	Gets HPC stall margin
getComponent(): Component	Gets associated component
setT24(Double): void	Sets T24 temperature
getT48(): Double	Gets T48 temperature
getNc(): Double	Gets core speed
setComponent(Component): void	Sets associated component
prePersist(): void	Pre-persistence operations
calculateHealthIndex(String): Map<String, Object>	Calculates health index

DataDTO Class

Attributes	
unit: String	Data unit
value: String	Data value
name: String	Data name
id: Integer	Data identifier
health: Integer	Health status
Methods	
getId(): Integer	Gets data ID
getName(): String	Gets data name
getValue(): String	Gets data value
getUnit(): String	Gets data unit

builder(): DataDTOBuilder	Creates builder
getHealth(): Integer	Gets health status
setId(Integer): void	Sets data ID
setHealth(Integer): void	Sets health status
setName(String): void	Sets data name
setValue(String): void	Sets data value
setUnit(String): void	Sets data unit
createMetricCardDTO(String, Double, String, Integer): DataDTO	Creates metric card DTO
fromData(Data): DataDTO	Creates DTO from Data entity
createAttributeDTO(String, Double): DataDTO	Creates attribute DTO

MonitorController Class

Attributes	
dataService: DataService	Data service instance
Methods	
getDeviceMetricCards(Integer, int, int): ResponseEntity<Map<String, Object>>	Gets device metric cards
getDeviceEnergyData(Integer): ResponseEntity<Map<String, Object>>	Gets device energy data
getDeviceHealthData(Integer): ResponseEntity<Map<String, Object>>	Gets device health data

DataService Interface

Methods	
getDeviceMetricCards(Integer, int, int): ReportDTO	Gets device metric cards
getDeviceEnergyData(Integer): ReportDTO	Gets device energy data
getDeviceHealthData(Integer): List<Double>	Gets device health data

DataServiceImpl Class

Attributes	
componentRepository: ComponentRepository	Component repository
dataRepository: DataRepository	Data repository
Methods	
getDeviceHealthData(Integer): List<Double>	Implementation of health data retrieval

getDeviceEnergyData(Integer): ReportDTO	Implementation of energy data retrieval
getDeviceMetricCards(Integer, int, int): ReportDTO	Implementation of metric cards
getLast7Days(): List<LocalDate>	Gets last 7 days
roundToTwoDecimals(Double): Double	Rounds to two decimal places
getOrderedValuesByDate (Map<LocalDate, Double>): List<Double>	Orders values by date
getComponentEnergyByDate(Integer): Map<LocalDate, Double>	Gets energy by date
getComponentHealthByDate(Integer): Map<LocalDate, Double>	Gets health by date

DataRepository Interface

Methods	
findFirstByComponentIdOrderByTimeDesc(Integer): Optional<Data>	Gets first data by component
getComponentMetricData(Integer, Pageable): Page<Data>	Gets metric data with pagination
getLatestComponentData(Integer): Optional<Data>	Gets latest component data
getComponentAttributes(Integer): List<Map<String, Object>>	Gets component attributes

5.4.4 Model System

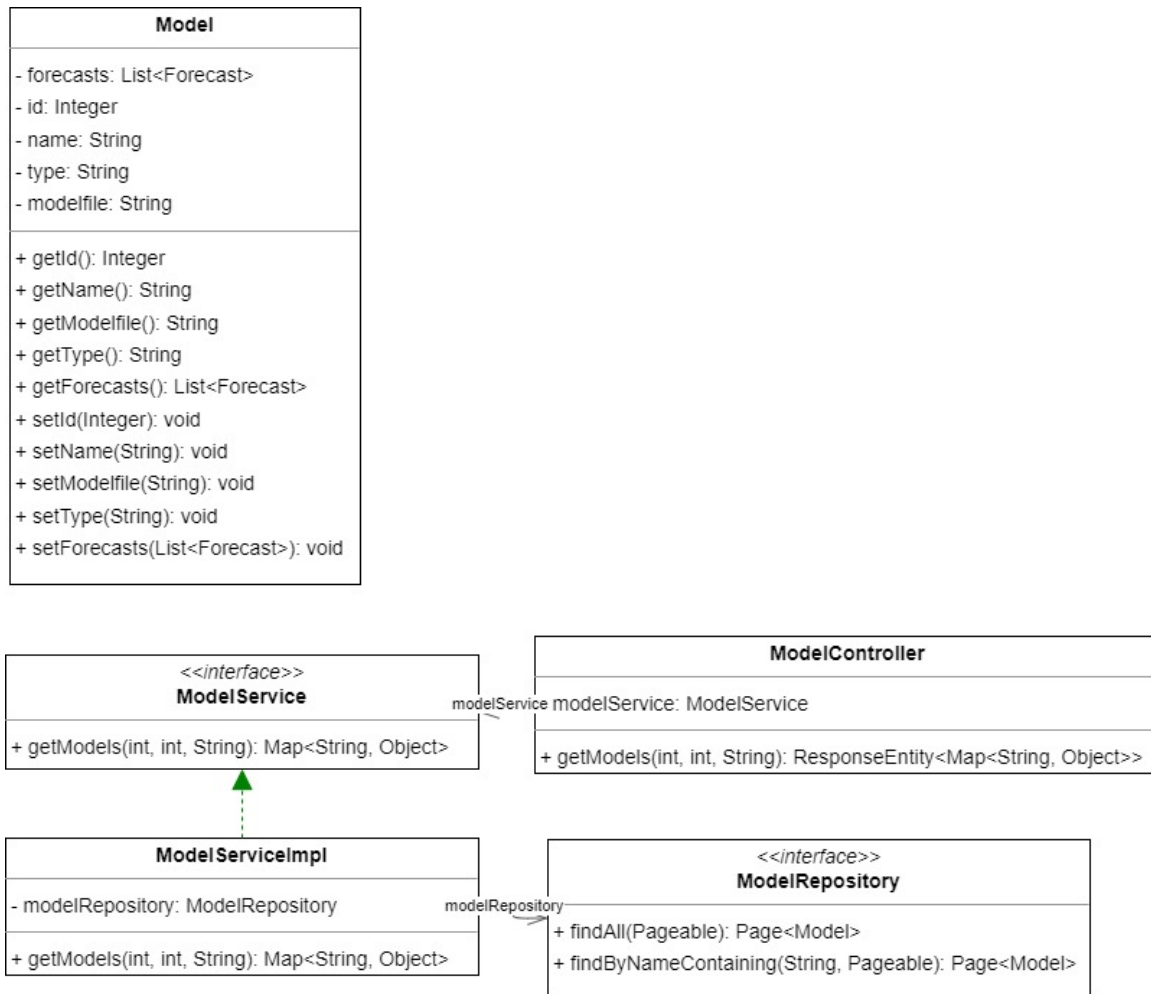


Figure 5.4: Model System Class Diagram

Model Entity

Attributes	
forecasts: List<Forecast>	List of forecasts generated by this model
id: Integer	Unique identifier for the model
name: String	Name of the model
type: String	Type/category of the model
modelfile: String	Path to the model file
Methods	
getId(): Integer	Gets model ID
getName(): String	Gets model name
getModelfile(): String	Gets model file path
getType(): String	Gets model type
getForecasts(): List<Forecast>	Gets associated forecasts
setId(Integer): void	Sets model ID
setName(String): void	Sets model name
setModelfile(String): void	Sets model file path
setType(String): void	Sets model type

setForecasts(List<Forecast>): void	Sets forecasts list
------------------------------------	---------------------

ModelController Class

Attributes	
modelService: ModelService	Model service instance
Methods	
getModels(int, String): ResponseEntity<Map<String, Object>>	Gets models with pagination

ModelService Interface

Methods	
getModels(int, String): Map<String, Object>	Gets models with filtering

ModelServiceImpl Class

Attributes	
modelRepository: ModelRepository	Model repository
Methods	
getModels(int, String): Map<String, Object>	Implementation of model retrieval

ModelRepository Interface

Methods	
findAll(Pageable): Page<Model>	Finds all models with pagination
findByNameContaining(String, Pageable): Page<Model>	Finds models by name

5.4.5 Report System



Figure 5.5: Report System Class Diagram

ReportController Class

Attributes	
reportService: ReportService	Report service instance
Methods	
getTodayAlertStats(): ResponseEntity<Map<String, Object>>	Gets today's alert statistics
exportDeviceAttributes(Integer): ResponseEntity<Resource>	Exports device attributes
getAllAlertStats(): ResponseEntity<Map<String, Object>>	Gets all alert statistics

getWeeklyAlertStats(): ResponseEntity<Map<String, Object>>	Gets weekly alert statistics
getDeviceAttributes(Integer): ResponseEntity<Map<String, Object>>	Gets device attributes

ReportService Interface

Methods	
exportDeviceAttributes(Integer): Resource	Exports device attributes to file
getTodayAlertStats(Integer): Map<String, Long>	Gets today's alert statistics
getAllAlertStats(Integer): Map<String, Long>	Gets all alert statistics
getWeeklyAlertStats(Integer): ReportDTO	Gets weekly alert statistics
getDeviceAttributes(Integer): List<DataDTO>	Gets device attributes

ReportServiceImpl Class

Attributes	
componentRepository: ComponentRepository	Component repository
alertRepository: AlertRepository	Alert repository
dataRepository: DataRepository	Data repository
Methods	
setCellValue(Cell, Double): void	Sets cell value in Excel
getAllAlertStats(Integer): Map<String, Long>	Implementation of alert stats
getTodayAlertStats(Integer): Map<String, Long>	Implementation of today's stats
exportDeviceAttributes(Integer): Resource	Implementation of attribute export
getWeeklyAlertStats(Integer): ReportDTO	Implementation of weekly stats
getDeviceAttributes(Integer): List<DataDTO>	Implementation of device attributes
roundToTwoDecimals(Double): Double	Rounds numbers to two decimals

ReportDTO Class

Attributes	
confirmedToday: long	Today's confirmed alerts
confirmedWeekly: long	This week's confirmed alerts

unconfirmedToday: long	Today's unconfirmed alerts
totalPages: int	Total pages for pagination
dailyStats: List<DailyStatDTO>	Daily statistics list
values: List<Double>	Numeric values list
dateValues: Map<String, Double>	Date-value mapping
unconfirmedWeekly: long	Weekly unconfirmed alerts
confirmed: long	Total confirmed alerts
unconfirmed: long	Total unconfirmed alerts
statusSummary: List<Map<String, Object>>	Status summary data
totalWeekly: long	Total weekly alerts
items: List<DataDTO>	Data items list
totalAlerts: long	Total number of alerts
energyCost: Double	Energy cost calculation
Methods	
getUnconfirmed(): long	Gets unconfirmed count
setConfirmedToday(long): void	Sets today's confirmed count
setDateValues(Map<String, Double>): void	Sets date-value pairs
setConfirmedWeekly(long): void	Sets weekly confirmed count
setTotalPages(int): void	Sets total page count
builder(): ReportDTOBuilder	Creates builder pattern instance
setItems(List<DataDTO>): void	Sets data items
getValues(): List<Double>	Gets values list
getConfirmedWeekly(): long	Gets weekly confirmed count
setStatusSummary(List<Map<String, Object>>): void	Sets status summary
getUnconfirmedWeekly(): long	Gets weekly unconfirmed
getStatusSummary(): List<Map<String, Object>>	Gets status summary
getUnconfirmed(): long	Gets unconfirmed alerts
getConfirmed(): long	Gets confirmed alerts
getTotalAlerts(): long	Gets total alert count
getTotalWeekly(): long	Gets weekly total
getEnergyCost(): Double	Gets energy cost
getDailyStats(): List<DailyStatDTO>	Gets daily statistics
toString(): String	String representation
getItems(): List<DataDTO>	Gets data items
getUnconfirmedToday(): long	Gets today's unconfirmed
getConfirmedToday(): long	Gets today's confirmed
getTotalPages(): int	Gets total pages
setUnconfirmedToday(long): void	Sets today's unconfirmed
setTotalAlerts(long): void	Sets total alerts
setTotalWeekly(long): void	Sets weekly total
setEnergyCost(Double): void	Sets energy cost

setDailyStats(List<DailyStatDTO>): void	Sets daily statistics
setUnconfirmedWeekly(long): void	Sets weekly unconfirmed
setConfirmed(long): void	Sets confirmed count
setUnconfirmed(long): void	Sets unconfirmed count
equals(Object): boolean	Equality comparison
hashCode(): int	Hash code generation
canEqual(Object): boolean	Equality helper

5.4.6 Simulation System

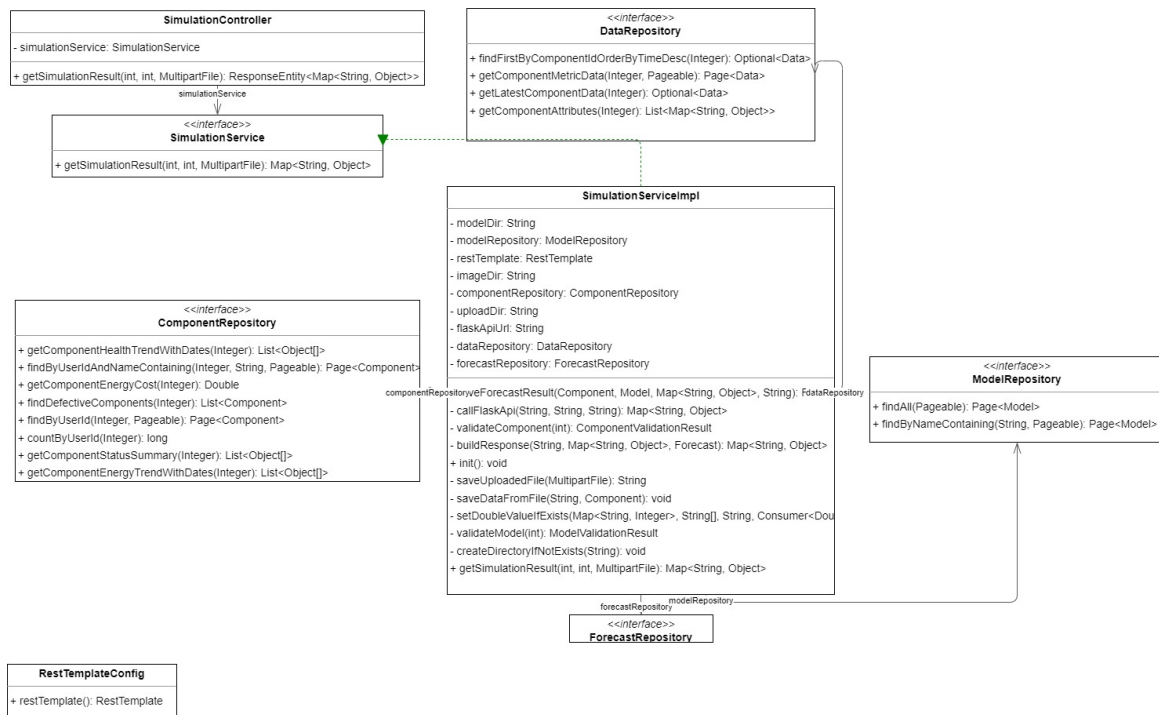


Figure 5.6: Simulation System Class Diagram

SimulationController Class

Attributes		
simulationService:	SimulationService	Simulation service instance
Methods		
getSimulationResult(int, int, MultipartFile):	ResponseEntity<Map<String, Object>>	Gets simulation results

SimulationService Interface

Methods	
getSimulationResult(int, int, MultipartFile):	Map<String, Object> Processes simulation request

SimulationServiceImpl Class

Attributes	
modelDir: String	Directory for model files
modelRepository: ModelRepository	Model repository
restTemplate: RestTemplate	REST client template
imageDir: String	Directory for image files
componentRepository: ComponentRepository	Component repository
uploadDir: String	Upload directory path
flaskApiUrl: String	Flask API URL
dataRepository: DataRepository	Data repository
forecastRepository: ForecastRepository	Forecast repository
Methods	
saveForecastResult(Component, Model, Map<String, Object>, String): void	Saves forecast results
callFlaskApi(String, String, String): Map<String, Object>	Calls Flask API
validateComponentId(int): ComponentValidationResult	Validates component ID
buildResponse(String, Map<String, Object>, Forecast): Map<String, Object>	Builds response object
init(): void	Initialization method
saveUploadedFile(MultipartFile): String	Saves uploaded file
saveDataFromFile(String, Component): void	Saves data from file
setDoubleValueIfExists(Map<String, Integer>, String[], String, Consumer<Double>): void	Sets double value conditionally
validateModel(int): ModelValidationResult	Validates model
createDirectoryIfNotExists(String): void	Creates directory if needed
getSimulationResult(int, int, MultipartFile): Map<String, Object>	Implementation of simulation

RestTemplateConfig Class

Methods	
restTemplate(): RestTemplate	Creates RestTemplate bean

5.4.7 User System

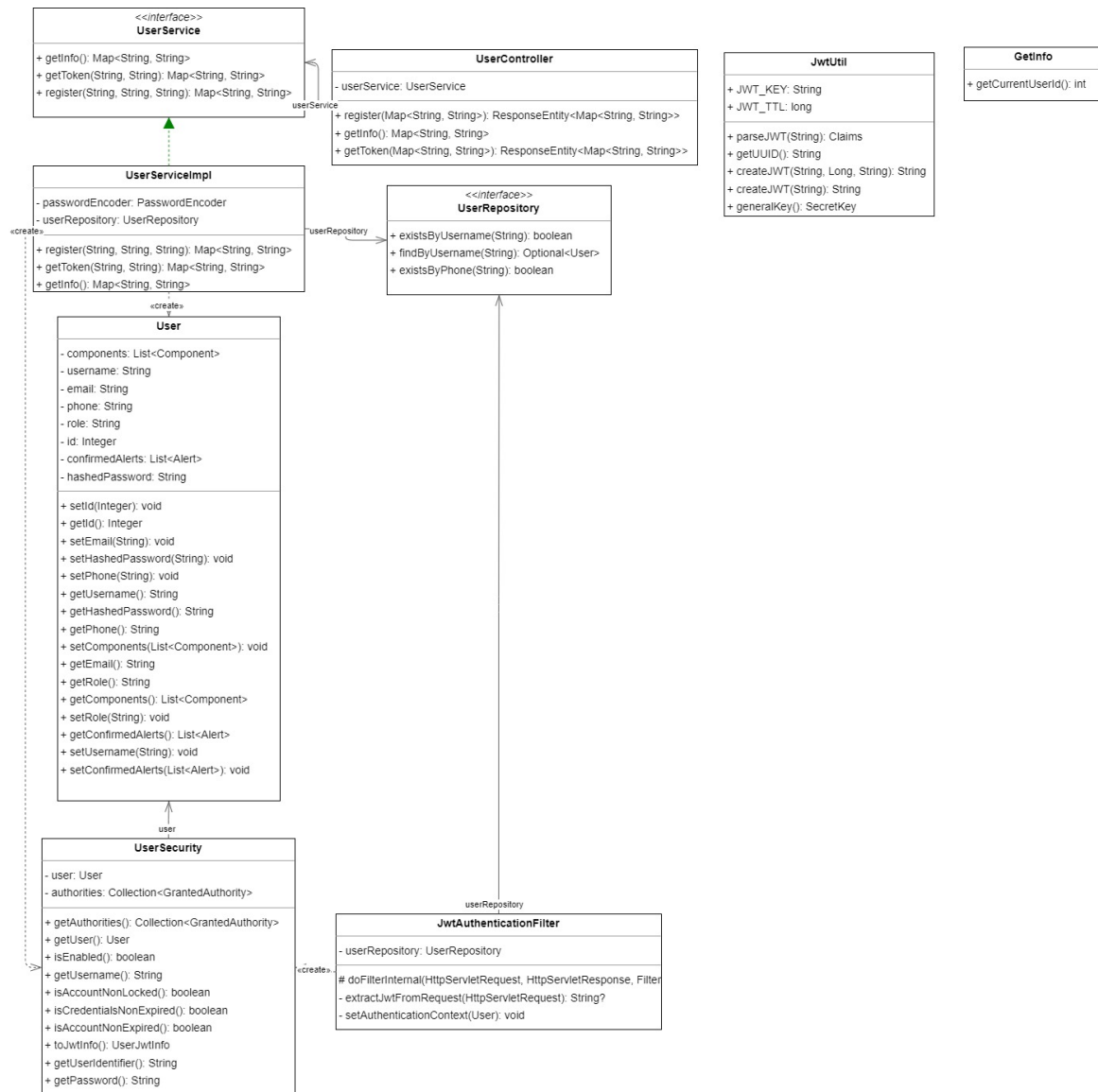


Figure 5.7: User System Class Diagram

User Entity

Attributes	
components: List<Component>	Components owned by user
username: String	User login name
email: String	User email address
phone: String	User phone number
role: String	User role (admin, user, etc.)
id: Integer	Unique user identifier
confirmedAlerts: List<Alert>	Alerts confirmed by user
hashedPassword: String	Encrypted password
Methods	
setId(Integer): void	Sets user ID

getId(): Integer	Gets user ID
setEmail(String): void	Sets email address
setHashedPassword(String): void	Sets hashed password
setPhone(String): void	Sets phone number
getUsername(): String	Gets username
getHashedPassword(): String	Gets hashed password
getPhone(): String	Gets phone number
setComponents(List<Component>): void	Sets components list
getEmail(): String	Gets email address
getComponents(): List<Component>	Gets components list
setRole(String): void	Sets user role
getRole(): String	Gets user role
getConfirmedAlerts(): List<Alert>	Gets confirmed alerts
setUsername(String): void	Sets username
setConfirmedAlerts(List<Alert>): void	Sets confirmed alerts

UserController Class

Attributes	
userService: UserService	User service instance
Methods	
register(Map<String, String>: ResponseEntity<Map<String, String>>	User registration
getInfo(): Map<String, String>	Gets user information
getToken(Map<String, String>: ResponseEntity<Map<String, String>>	User login

UserService Interface

Methods	
getInfo(): Map<String, String>	Gets user information
getToken(String, String): Map<String, String>	Authenticates and returns token
register(String, String, String): Map<String, String>	Registers new user

UserServiceImpl Class

Attributes	
passwordEncoder: PasswordEncoder	Password encoding service
userRepository: UserRepository	User data repository

Methods	
register(String, String, String): Map<String, String>	Implementation of user registration
getToken(String, String): Map<String, String>	Implementation of authentication
getInfo(): Map<String, String>	Implementation of user info retrieval

UserRepository Interface

Methods	
existsByUsername(String): boolean	Checks if username exists
findByUsername(String): Optional<User>	Finds user by username
existsByPhone(String): boolean	Checks if phone exists

JwtAuthenticationFilter Class

Attributes	
userRepository: UserRepository	User repository
Methods	
doFilterInternal(HttpServletRequest, HttpServletResponse, FilterChain): void	JWT filter logic
extractJwtFromRequest(HttpServletRequest): String?	Extracts JWT from request
setAuthenticationContext(User): void	Sets authentication context

JwtUtil Class

Attributes	
JWT_KEY: String	JWT signing key
JWT_TTL: long	JWT time to live
Methods	
parseJWT(String): Claims	Parses JWT token
getUUID(): String	Gets UUID for token
createJWT(String, Long, String): String	Creates JWT token
createJWT(String): String	Creates JWT with default TTL
generateKey(): SecretKey	Generates signing key

GetInfo Class

Methods	
getCurrentUserId(): int	Gets current user ID

5 Class Relationships

The class design follows an object-oriented approach with clear relationships between classes:

1. Inheritance Relationships:

- Service implementations implement their corresponding service interfaces
- Repository interfaces extend JpaRepository for data access operations
- Controllers follow REST controller patterns

2. Composition Relationships:

- User entity contains lists of Components and confirmed Alerts
- Component entity contains lists of Data, Alerts, and Forecasts
- Model entity contains lists of Forecasts

3. Dependency Relationships:

- Controllers depend on Services for business logic execution
- Services depend on Repositories for data persistence operations
- DTOs are used for data transfer between layers
- Service implementations utilize multiple repositories for complex operations

4. Association Relationships:

- Alert is associated with Component and User (confirming user)
- Data is associated with Component for sensor readings
- Forecast is associated with Component and Model

This design promotes separation of concerns, modularity, and maintainability while ensuring clear communication patterns between different layers of the application architecture.

Chapter6 Data Design

1 Data Models

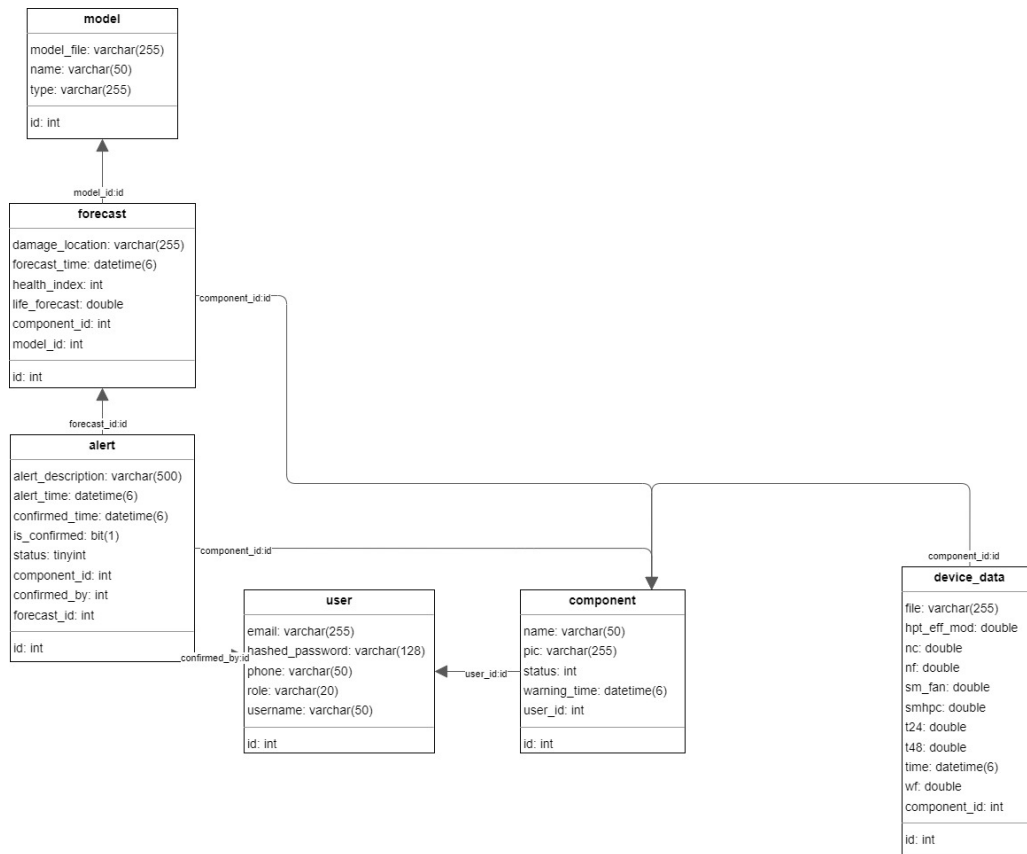


Figure 6.1: Database Entity Relationship Diagram

The system's main data entities are designed to support comprehensive aero engine maintenance operations. Based on the Entity Relationship Diagram, the core entities include:

1. User

- id: int (Primary Key)
- email: varchar(255)
- hashed_password: varchar(128)
- phone: varchar(50)
- role: varchar(20)
- username: varchar(50)

2. Component

- id: int (Primary Key)
- name: varchar(50)
- pic: varchar(255)

- status: int
- warning_time: datetime(6)
- user_id: int (Foreign Key)

3. Device_Data

- id: int (Primary Key)
- file: varchar(255)
- hpt_eff_mod: double
- nc: double
- nf: double
- sm_fan: double
- smhpc: double
- t24: double
- t48: double
- time: datetime(6)
- wf: double
- component_id: int (Foreign Key)

4. Model

- id: int (Primary Key)
- model_file: varchar(255)
- name: varchar(50)
- type: varchar(255)

5. Forecast

- id: int (Primary Key)
- damage_location: varchar(255)
- forecast_time: datetime(6)
- health_index: int
- life_forecast: double
- component_id: int (Foreign Key)
- model_id: int (Foreign Key)

6. Alert

- id: int (Primary Key)
- alert_description: varchar(500)
- alert_time: datetime(6)
- confirmed_time: datetime(6)
- is_confirmed: bit(1)
- status: tinyint
- component_id: int (Foreign Key)
- confirmed_by: int (Foreign Key)
- forecast_id: int (Foreign Key)

2 Entity Relationships

The database design implements several key relationships that support the system's operational requirements:

1. User-Component Relationship

- One-to-Many: Each user can own multiple components
- Foreign Key: `component.user_id` references `user.id`
- Supports multi-user component management

2. Component-Device Data Relationship

- One-to-Many: Each component can have multiple sensor data records
- Foreign Key: `device_data.component_id` references `component.id`
- Enables comprehensive data collection for each component

3. Component-Forecast Relationship

- One-to-Many: Each component can have multiple forecast predictions
- Foreign Key: `forecast.component_id` references `component.id`
- Supports historical prediction tracking

4. Model-Forecast Relationship

- One-to-Many: Each model can generate multiple forecasts
- Foreign Key: `forecast.model_id` references `model.id`
- Tracks which model generated each prediction

5. Component-Alert Relationship

- One-to-Many: Each component can trigger multiple alerts
- Foreign Key: `alert.component_id` references `component.id`
- Enables comprehensive alert management per component

6. User-Alert Relationship

- One-to-Many: Each user can confirm multiple alerts
- Foreign Key: `alert.confirmed_by` references `user.id`
- Tracks alert confirmation responsibility

7. Forecast-Alert Relationship

- One-to-Many: Each forecast can trigger multiple alerts
- Foreign Key: `alert.forecast_id` references `forecast.id`
- Links predictive analysis with alert generation

3 Database Architecture Design

The system adopts a structured database architecture optimized for industrial maintenance operations:

1. Relational Database (MySQL)

- Primary storage for all structured data entities
- Supports ACID transactions for data consistency
- Implements proper indexing on foreign keys and frequently queried fields
- Uses normalized table structure to minimize data redundancy
- Supports complex queries for reporting and analytics

2. Data Integrity Constraints

- Foreign key constraints ensure referential integrity
- NOT NULL constraints on critical fields
- Check constraints for data validation
- Unique constraints on business keys

3. Performance Optimization

- Indexes on primary and foreign keys
- Composite indexes for multi-column queries
- Partitioning strategies for large tables (device_data)
- Query optimization for time-series data access

4. Data Storage Considerations

- Efficient storage of sensor data with appropriate data types
- BLOB storage for model files through file system integration
- Timestamp precision for accurate time-series analysis
- Varchar sizing optimized for actual data requirements

4 Data Access Layer

The data access layer implements the repository pattern to provide standardized database operations:

1. Repository Interfaces

- UserRepository: User management operations
- ComponentRepository: Component lifecycle management
- DataRepository: Sensor data operations
- ModelRepository: AI model metadata management
- ForecastRepository: Prediction result storage
- AlertRepository: Alert lifecycle management

2. Data Transfer Objects (DTOs)

- Lightweight objects for data transfer between layers
- Validation annotations for input data verification

- Builder patterns for complex object construction
- Conversion utilities between entities and DTOs

3. Transaction Management

- Declarative transaction management using Spring annotations
- Rollback strategies for failed operations
- Isolation levels appropriate for concurrent access
- Bulk operations for performance optimization

This database design ensures data consistency, supports complex business requirements, and provides the foundation for reliable industrial equipment maintenance operations.

Chapter7 Module Designs

1 Device Center Module

7.1.1 Purpose and Functionality

The Device Center Module serves as the central repository for all aero engine component information within the Smart Maintenance Platform. Based on the system's actual implementation, it provides:

- Comprehensive component catalog management for user-owned devices
- Real-time health status monitoring and assessment
- Device status distribution analytics and reporting
- Component identification and basic information retrieval
- Integration with predictive maintenance and alert systems

7.1.2 Module Components

1. Component Management Service

- Functionality: Manages component lifecycle and user associations
- Interfaces:
 - `getUserDevices()`: Retrieves user's components with search and pagination
 - `getUserDeviceCount()`: Gets total device count for user
 - `getDeviceById()`: Retrieves specific device information
 - `getUserDefectiveDevices()`: Identifies components requiring attention

2. Status Analysis Engine

- Functionality: Analyzes and categorizes device health status
- Interfaces:
 - `getUserDeviceStatusSummary()`: Generates status distribution analytics
 - `calculateHealthMetrics()`: Computes component health indicators
 - `identifyDefectiveComponents()`: Flags components with abnormal status

3. Data Integration Layer

- Functionality: Integrates with sensor data and prediction systems
- Interfaces:
 - `getComponentData()`: Retrieves associated sensor readings
 - `getHealthTrends()`: Provides historical health progression
 - `getEnergyConsumption()`: Monitors energy usage patterns

7.1.3 Data Flow

1. User requests device information through the ComponentController
2. ComponentService retrieves component data from ComponentRepository
3. System processes real-time sensor data to update health metrics
4. Status analysis engine categorizes components by health status
5. Processed device information is returned with current status indicators
6. Status changes trigger notifications to monitoring and alert systems

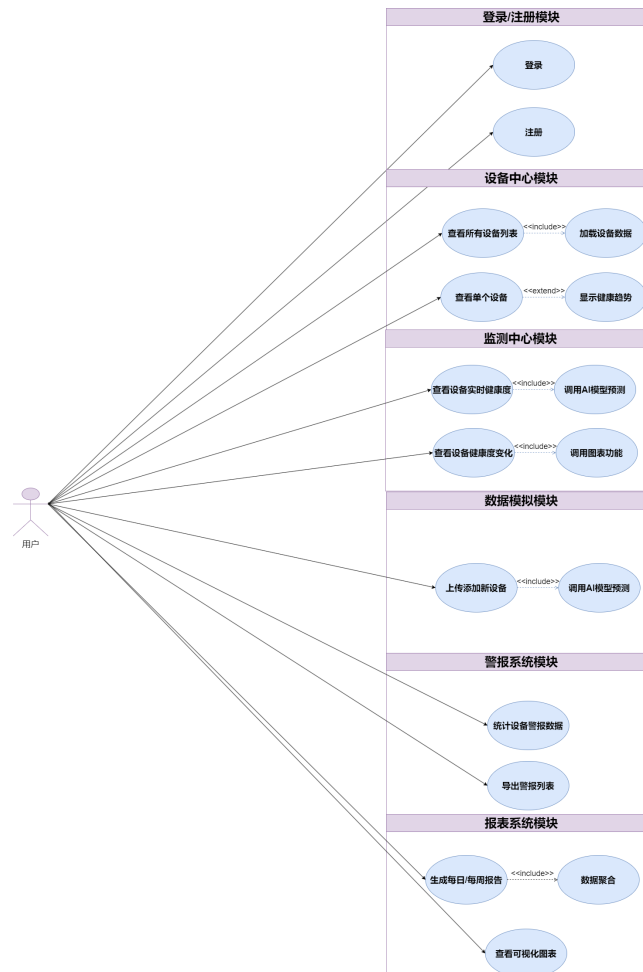


Figure 7.1: Use Case Diagram

2 Monitoring Center Module

7.2.1 Purpose and Functionality

The Monitoring Center Module provides comprehensive real-time and historical monitoring capabilities for aero engine components. It delivers:

- Real-time health data visualization and analysis
- Energy consumption monitoring with cost calculations
- Component metric cards for detailed parameter tracking

- Historical trend analysis for performance assessment
- Integration with predictive analytics for proactive maintenance

7.2.2 Module Components

1. Real-time Data Processor

- Functionality: Processes and validates incoming sensor data streams
- Interfaces:
 - `getDeviceHealthData()`: Retrieves current health metrics
 - `processHealthIndex()`: Calculates component health scores
 - `validateSensorReadings()`: Ensures data quality and integrity

2. Energy Monitoring Engine

- Functionality: Tracks energy consumption and calculates costs
- Interfaces:
 - `getDeviceEnergyData()`: Retrieves energy consumption patterns
 - `calculateEnergyCost()`: Computes operational costs
 - `analyzeEnergyTrends()`: Identifies consumption patterns

3. Metric Card Generator

- Functionality: Creates comprehensive metric displays for components
- Interfaces:
 - `getDeviceMetricCards()`: Generates paginated metric displays
 - `formatMetricData()`: Formats sensor data for visualization
 - `calculateHealthIndicators()`: Computes component-specific metrics

4. Trend Analysis Engine

- Functionality: Analyzes historical patterns and performance trends
- Interfaces:
 - `getHealthTrendData()`: Provides health progression analysis
 - `getEnergyTrendData()`: Tracks energy consumption over time
 - `identifyPerformancePatterns()`: Detects operational patterns

7.2.3 Data Flow

1. MonitorController receives requests for device monitoring data
2. DataService retrieves latest sensor readings from DataRepository
3. Real-time processor validates and formats sensor data
4. Health metrics are calculated using component-specific algorithms
5. Energy consumption data is processed and cost calculations performed
6. Formatted monitoring data is returned to the user interface
7. Trend analysis provides historical context for current readings

3 Simulation and Prediction Module

7.3.1 Purpose and Functionality

The Simulation and Prediction Module leverages advanced AI models to provide predictive maintenance capabilities. It enables:

- Remaining Useful Life (RUL) prediction using CNN-LSTM models
- Health status forecasting with confidence intervals
- Damage location identification through image analysis
- What-if scenario analysis for maintenance planning
- Model performance evaluation and comparison

7.3.2 Module Components

1. Model Management System

- Functionality: Manages AI models and their metadata
- Interfaces:
 - `getModels()`: Retrieves available prediction models
 - `validateModel()`: Ensures model integrity and compatibility
 - `loadModelMetadata()`: Retrieves model specifications

2. Simulation Execution Engine

- Functionality: Executes predictive simulations using external AI services
- Interfaces:
 - `getSimulationResult()`: Processes simulation requests
 - `callFlaskApi()`: Interfaces with external AI prediction services
 - `processUploadedData()`: Handles input data preprocessing

3. Results Analysis Engine

- Functionality: Analyzes and interprets prediction results
- Interfaces:
 - `saveForecastResult()`: Stores prediction outcomes
 - `calculateHealthIndex()`: Computes health indicators
 - `identifyDamageLocation()`: Locates potential failure points

4. Data Processing Pipeline

- Functionality: Prepares and validates input data for models
- Interfaces:
 - `saveDataFromFile()`: Processes uploaded sensor data
 - `validateComponentData()`: Ensures data compatibility
 - `normalizeInputs()`: Standardizes data for model consumption

7.3.3 Data Flow

1. User uploads component data through SimulationController
2. System validates component and model compatibility
3. Input data is preprocessed and saved to the system
4. External AI service (Flask API) receives processed data for analysis
5. AI models generate predictions including RUL and health assessments
6. Results are processed and stored in the forecast repository
7. Comprehensive simulation results are returned with visual feedback
8. Alert system is notified if predictions indicate critical conditions

4 Alert System Module

7.4.1 Purpose and Functionality

The Alert System Module provides comprehensive alert management for the maintenance platform. It delivers:

- Real-time alert generation based on component status and predictions
- Multi-level alert categorization with severity indicators
- Batch alert processing for efficient management
- Alert confirmation tracking and user accountability
- Historical alert analysis and reporting capabilities

7.4.2 Module Components

1. Alert Generation Engine

- Functionality: Creates alerts based on system conditions and predictions
- Interfaces:
 - `generateAlert()`: Creates new alerts from system triggers
 - `classifyAlertSeverity()`: Determines alert priority levels
 - `linkToForecast()`: Associates alerts with prediction results

2. Alert Management System

- Functionality: Manages alert lifecycle and user interactions
- Interfaces:
 - `getUnconfirmedAlerts()`: Retrieves alerts requiring attention
 - `confirmAlerts()`: Processes alert confirmations
 - `deleteAlerts()`: Removes resolved or invalid alerts

3. Alert Analytics Engine

- Functionality: Provides statistical analysis of alert patterns
- Interfaces:

- `getAlertsSummary()` : Generates alert distribution statistics
- `analyzeAlertTrends()` : Identifies patterns in alert generation
- `calculateResponseMetrics()` : Measures alert response performance

4. Export and Reporting System

- Functionality: Enables alert data export and documentation
- Interfaces:
 - `exportAlertsToXLSX()` : Generates Excel reports of alerts
 - `formatAlertData()` : Prepares data for external systems
 - `generateAlertReports()` : Creates comprehensive alert summaries

7.4.3 Data Flow

1. System continuously monitors component status and prediction results
2. Alert generation engine evaluates conditions against defined thresholds
3. New alerts are created and linked to components and forecasts
4. `AlertController` processes user requests for alert information
5. Users can confirm, delete, or export alerts through the interface
6. Alert status changes are tracked with user accountability
7. Analytics engine provides insights into alert patterns and trends

5 Reporting System Module

7.5.1 Purpose and Functionality

The Reporting System Module generates comprehensive analytics and insights from system data. It provides:

- Real-time and historical alert statistics and trends
- Device attribute reporting with export capabilities
- Weekly and daily operational summaries
- Maintenance effectiveness analytics
- Customizable report generation and distribution

7.5.2 Module Components

1. Alert Statistics Engine

- Functionality: Generates comprehensive alert analytics
- Interfaces:
 - `getTodayAlertStats()` : Provides daily alert summaries
 - `getAllAlertStats()` : Generates comprehensive alert statistics
 - `getWeeklyAlertStats()` : Creates weekly trend analysis

2. Device Attribute Analyzer

- **Functionality:** Analyzes and reports device characteristics and performance
- **Interfaces:**
 - `getDeviceAttributes()`: Retrieves component attribute data
 - `exportDeviceAttributes()`: Generates Excel exports
 - `analyzeAttributePatterns()`: Identifies performance patterns

3. Data Aggregation Engine

- **Functionality:** Processes and aggregates data from multiple sources
- **Interfaces:**
 - `aggregateAlertData()`: Combines alert information
 - `calculateStatistics()`: Computes statistical measures
 - `generateTimeSeries()`: Creates temporal data analysis

4. Report Export System

- **Functionality:** Formats and exports reports in various formats
- **Interfaces:**
 - `generateExcelReport()`: Creates Excel-formatted reports
 - `formatReportData()`: Structures data for presentation
 - `createVisualizationData()`: Prepares data for charts and graphs

7.5.3 Data Flow

1. ReportController receives requests for specific report types
2. ReportService queries relevant repositories for required data
3. Data aggregation engine processes and combines information from multiple sources
4. Statistical calculations are performed to generate meaningful insights
5. Report formatting prepares data for presentation or export
6. Generated reports are returned to users or exported to files
7. Historical data is maintained for trend analysis and comparison

6 User Management Module

7.6.1 Purpose and Functionality

The User Management Module handles all aspects of user authentication, authorization, and profile management. It provides:

- Secure user authentication using JWT tokens
- User registration with validation and verification
- Profile information management and retrieval
- Session management and security enforcement
- Integration with component ownership and access control

7.6.2 Module Components

1. Authentication Service

- **Functionality:** Manages user login and token generation
- **Interfaces:**
 - `getToken()`: Authenticates users and issues JWT tokens
 - `validateCredentials()`: Verifies username and password
 - `generateSecureToken()`: Creates cryptographically secure tokens

2. User Registration System

- **Functionality:** Handles new user account creation
- **Interfaces:**
 - `register()`: Creates new user accounts
 - `validateUserData()`: Ensures data integrity and uniqueness
 - `encryptPassword()`: Securely stores user credentials

3. Profile Management Service

- **Functionality:** Manages user profile information and preferences
- **Interfaces:**
 - `getInfo()`: Retrieves current user information
 - `updateProfile()`: Modifies user account details
 - `managePreferences()`: Handles user configuration settings

4. Security Enforcement Layer

- **Functionality:** Implements security policies and access control
- **Interfaces:**
 - `validateJWTToken()`: Verifies token authenticity
 - `enforceAccessControl()`: Manages resource permissions
 - `auditUserActivity()`: Logs security-relevant events

7.6.3 Data Flow

1. UserController receives authentication or registration requests
2. UserService validates input data and processes requests
3. Authentication service verifies credentials against stored data
4. JWT tokens are generated and returned for valid authentication
5. User information is retrieved and managed through profile services
6. Security layer enforces access control throughout the system
7. All user activities are logged for security and audit purposes

This modular design ensures clear separation of concerns while maintaining integration between components, supporting the system's goals of providing comprehensive predictive maintenance capabilities for aero engine industrial equipment.

Chapter8 Machine Learning Model Design

1 Model Selection Rationale

Based on comprehensive evaluation of various machine learning approaches, several models have been identified as suitable for the predictive maintenance tasks:

8.1.1 For Anomaly Detection

- **Skip-GANomaly**: Selected for its ability to effectively model normal behavior patterns and detect deviations. This model combines the strengths of skip connections with the generative-adversarial approach, making it particularly effective for complex machinery like aero engines.
- **PHOT (Phase-Only Transform)**: Selected for its capability to process frequency domain features, which is valuable for detecting vibration-related anomalies in rotating machinery.

8.1.2 For Remaining Useful Life Prediction

- **CNN-LSTM**: Selected for combining CNN's spatial feature extraction capabilities with LSTM's temporal modeling strengths. This hybrid approach has demonstrated superior performance in capturing both spatial and temporal patterns in sensor data.
- **CNN-Transformer**: Selected for capturing long-range dependencies in time series data, which is essential for understanding the complex degradation patterns of aero engine components.
- **DBN-BiGRU**: Selected for deep feature extraction and bidirectional temporal modeling, providing robust prediction capabilities across various operational conditions.

2 Skip-GANomaly Model Architecture

The Skip-GANomaly model is a semi-supervised anomaly detection approach that combines the power of skip connections with generative adversarial networks (GANs).

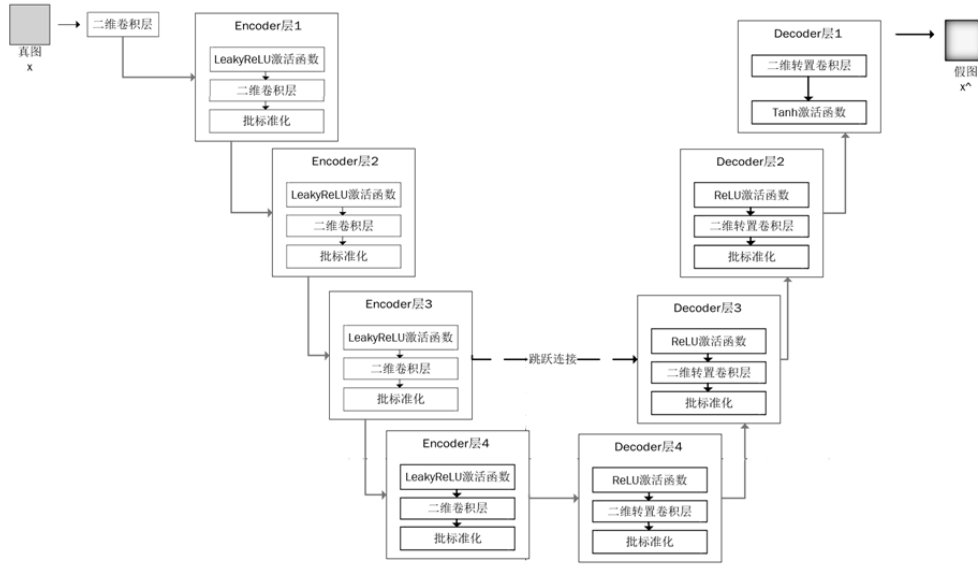


Figure 8.1: Skip-GANomaly Generator Architecture with Skip Connections

8.2.1 Model Structure

As illustrated in Figure 8.1, the Skip-GANomaly architecture consists of two main components:

- **Generator:** Encoder-decoder architecture with skip connections
 - Input layer: Accepts sensor data of shape (batch_size, sequence_length, feature_dim)
 - Encoder: Four convolutional layers with LeakyReLU activation and batch normalization
 - Latent space: Compressed representation of normal patterns at the bottleneck
 - Decoder: Four deconvolutional layers with ReLU activation and batch normalization
 - Skip connections: Direct connections between corresponding encoder and decoder layers
 - Output layer: Reconstructed sensor data with Tanh activation
- **Discriminator:** CNN-based binary classifier
 - Input layer: Accepts original or reconstructed sensor data
 - Feature extractor: Four convolutional layers with LeakyReLU activation
 - Classifier: Final convolutional layer with sigmoid activation
 - Output layer: Single probability score (0 for fake, 1 for real)

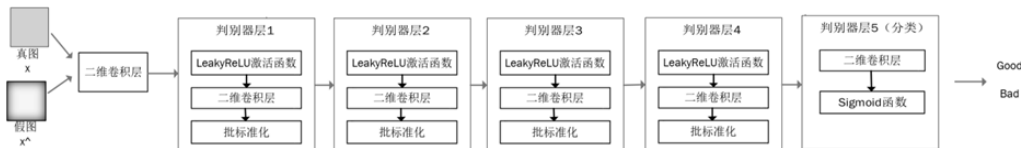


Figure 8.2: Skip-GANomaly Discriminator Network Structure

8.2.2 Training Approach

The training process employs multiple loss functions as shown in Figure 8.2:

- **Adversarial Loss (L_{adv}):** Encourages the generator to produce realistic reconstructions that fool the discriminator

- **Contextual Loss (L_{con}):** L1 distance between input and reconstructed output to ensure reconstruction fidelity
- **Latent Loss (L_{lat}):** L2 distance between intermediate features to match feature representations
- **Total Loss:** $L = \lambda_{adv}L_{adv} + \lambda_{con}L_{con} + \lambda_{lat}L_{lat}$
- **Training Data:** Only normal operational data used during training phase
- **Optimizer:** Adam optimizer with learning rate of 0.0002 and betas (0.5, 0.999)

8.2.3 Anomaly Detection Process

- **Anomaly Score Computation:** Weighted combination of reconstruction error and discriminator feature matching
- **Thresholding:** Dynamic threshold based on statistical properties of normal training data
- **Post-processing:** Moving average smoothing and peak detection for robust anomaly identification

3 CNN-LSTM Model Architecture

The CNN-LSTM model is specifically designed for remaining useful life (RUL) prediction tasks, combining the spatial feature extraction capabilities of convolutional neural networks with the temporal modeling strengths of long short-term memory networks.

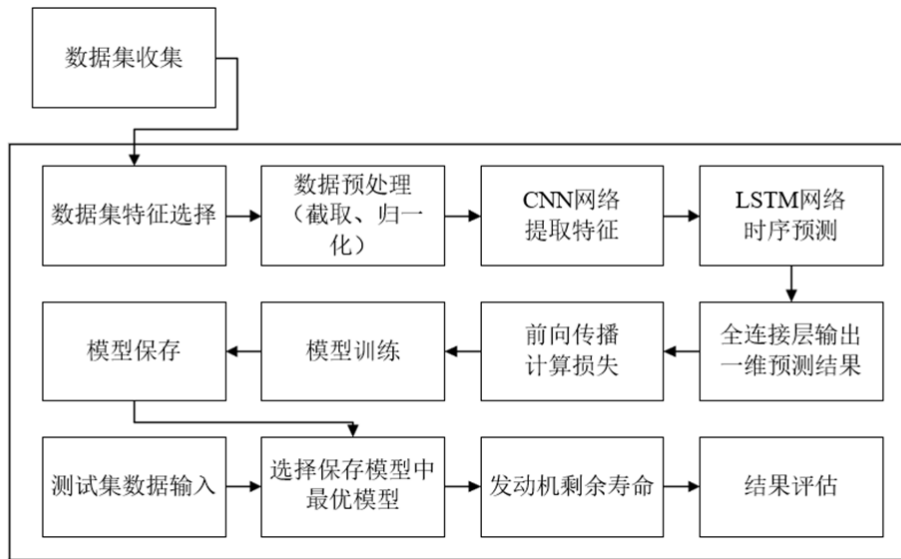


Figure 8.3: CNN-LSTM Hybrid Model Architecture for RUL Prediction

8.3.1 Model Structure

As depicted in Figure 8.3, the CNN-LSTM model consists of several integrated components:

- **Input Layer:** Accepts multivariate time series data of shape (batch_size, sequence_length, feature_dim)
- **CNN Module:** Extracts spatial features from sensor data
 - Three convolutional layers (32, 64, 128 filters) with kernel size 3
 - Batch normalization after each convolutional layer
 - ReLU activation functions
 - MaxPooling1D layers to reduce sequence length and extract dominant features

- **LSTM Module:** Captures temporal dependencies and sequential patterns
 - Bidirectional LSTM layer with 100 units for enhanced temporal modeling
 - Dropout (0.3) for regularization and overfitting prevention
 - Return sequences enabled for stacking multiple LSTM layers
 - Second LSTM layer with 50 units for refined temporal feature extraction
- **Fully Connected Module:** Final prediction head
 - Flatten layer to reshape LSTM outputs
 - Dense layer with 50 units and ReLU activation
 - Dropout (0.2) for additional regularization
 - Dense layer with 20 units and ReLU activation
 - Output layer with linear activation for RUL regression

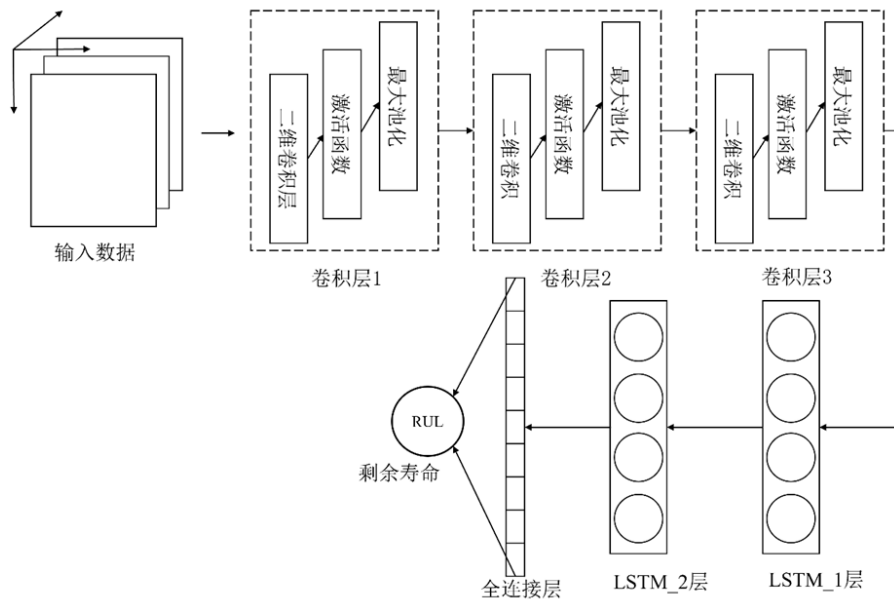


Figure 8.4: CNN-LSTM Data Flow and Processing Pipeline

8.3.2 Training Approach

The training methodology follows established best practices for time series prediction:

- **Loss Function:** Mean Squared Error (MSE) for regression task optimization
- **Evaluation Metrics:**
 - Root Mean Squared Error (RMSE) for error magnitude assessment
 - Mean Absolute Error (MAE) for average prediction deviation
 - R-squared for explained variance measurement
- **Optimizer:** Adam optimizer with initial learning rate of 0.001
- **Learning Rate Schedule:** ReduceLROnPlateau with patience of 5 epochs
- **Early Stopping:** Implemented with patience of 10 epochs to prevent overfitting
- **Training Data:** Historical sensor data with known RUL values from multiple engine units
- **Validation Strategy:** Time-based split maintaining chronological order

8.3.3 Model Evaluation and Performance Targets

Figure 8.4 illustrates the complete data processing pipeline and evaluation framework:

- **Performance Metrics and Targets:**
 - RMSE: Target < 10 operating hours for practical deployment
 - MAE: Target < 8 operating hours for maintenance planning accuracy
 - Prediction accuracy within $\pm 10\%$: Target > 85% for operational reliability
 - Convergence time: Target < 100 epochs for efficient training
- **Testing Methodology:**
 - Evaluation on held-out test sets from different engine units
 - Cross-validation across multiple operational scenarios
 - Robustness testing under various operating conditions
- **Comparative Analysis:**
 - Benchmarking against traditional methods (ARIMA, Random Forest)
 - Comparison with other deep learning approaches (vanilla LSTM, Transformer)
 - Performance analysis across different prediction horizons

4 Model Integration and Deployment Strategy

8.4.1 Model Selection Criteria

The selection of Skip-GANomaly for anomaly detection and CNN-LSTM for RUL prediction is based on:

- **Technical Performance:** Superior accuracy metrics compared to baseline methods
- **Computational Efficiency:** Reasonable inference time for real-time applications
- **Robustness:** Stable performance across different operational conditions
- **Interpretability:** Ability to provide meaningful insights for maintenance decisions

8.4.2 Implementation Considerations

- **Model Versioning:** Systematic tracking of model iterations and performance improvements
- **Hyperparameter Optimization:** Grid search and Bayesian optimization for optimal configurations
- **Model Monitoring:** Continuous performance tracking and drift detection in production
- **Retraining Strategy:** Periodic model updates with new operational data

This comprehensive machine learning model design ensures robust and accurate predictive maintenance capabilities for aero engine industrial equipment, supporting the system's primary objectives of reducing unplanned downtime and optimizing maintenance operations.

Chapter9 User Interface Design

1 Design Principles

The user interface design for the Smart Maintenance Platform follows these guiding principles:

- **Clarity:** Present information in a clear and unambiguous manner
- **Consistency:** Maintain consistent patterns and behaviors throughout the application
- **Efficiency:** Optimize for task completion with minimal steps
- **Responsiveness:** Ensure appropriate design across different device sizes
- **Accessibility:** Support users with different abilities and needs
- **Visual Hierarchy:** Guide attention to the most important elements

2 UI Components

9.2.1 Dashboard

The main dashboard provides an overview of the system status:

- **Health Status Cards:** Visual indicators of overall fleet health
- **Alert Summary:** Count and distribution of active alerts by severity
- **Performance Trends:** Key performance indicators over time
- **Maintenance Calendar:** Upcoming scheduled maintenance
- **Quick Navigation:** Direct access to frequently used features

9.2.2 Device Detail View

Detailed view of individual device information:

- **Device Identity:** Name, type, model, serial number, location
- **Current Status:** Operational state, health score, utilization
- **Sensor Readings:** Current values with visual indicators for normal/abnormal
- **Predicted RUL:** Remaining useful life estimates with confidence intervals
- **Maintenance History:** Timeline of past maintenance activities
- **Anomaly Detection:** Highlighted anomalies with explanatory information

9.2.3 Monitoring Views

Real-time and historical monitoring of engine performance:

- **Parameter Dashboards:** Multi-parameter display with configurable layouts
- **Time Series Charts:** Interactive charts with zoom, pan, and selection
- **Correlation Views:** Visualization of relationships between parameters
- **Heat Maps:** Spatial representation of parameter distributions
- **Threshold Visualizations:** Clear indication of normal operating ranges

9.2.4 Alert Management

Interface for handling system alerts:

- **Alert List:** Filterable list of all alerts with severity indicators
- **Alert Details:** Comprehensive information about each alert
- **Action Tools:** Options to acknowledge, escalate, or resolve alerts
- **Notification Settings:** Configuration for alert delivery preferences
- **History View:** Past alerts with resolution information

9.2.5 Reporting Interface

Tools for generating and viewing reports:

- **Report Builder:** Customizable report configuration
- **Template Gallery:** Pre-defined report templates
- **Output Format Selection:** PDF, Excel, CSV export options
- **Scheduling Tools:** Calendar-based report scheduling
- **Distribution Lists:** Recipient management for automated reports

3 Navigation Structure

The application uses a hierarchical navigation structure:

- **Top Level:** Main functional areas (Dashboard, Devices, Monitoring, Alerts, Reports, Administration)
- **Second Level:** Specific functions within each area
- **Tertiary Level:** Detailed views and configuration screens

Navigation is facilitated through:

- **Main Navigation:** Persistent sidebar with expandable sections
- **Breadcrumb Trail:** Clear path showing current location
- **Quick Actions:** Context-sensitive actions for common tasks
- **Search:** Global search function accessible from all screens

4 Responsive Design Strategy

The interface adapts to different screen sizes and devices:

- **Desktop:** Full-featured experience with multi-pane layouts
- **Tablet:** Optimized layouts with rearranged content
- **Mobile:** Focused interface with prioritized information

Responsive techniques include:

- **Fluid Grid Layouts:** Content adjusts to available space
- **Flexible Media:** Images and videos scale appropriately
- **Progressive Disclosure:** Complex information revealed progressively
- **Touch-Friendly Controls:** Larger tap targets on touch devices

Chapter10 Interface Specifications

1 API Design

The system implements RESTful APIs for integration and communication between components.

10.1.1 API Principles

- **Resource-Oriented:** APIs are organized around resources
- **Standard HTTP Methods:** GET, POST, PUT, DELETE used appropriately
- **Consistent URL Structure:** /api/[module]/[action]
- **JSON Format:** All responses use JSON format
- **Pagination:** Results are paginated for large collections
- **Filtering:** Support for query parameters to filter results
- **Error Handling:** Consistent error response format

10.1.2 Authentication and Authorization

- **Authentication:** JWT (JSON Web Tokens)
- **Token Validity:** Configurable duration
- **Authorization:** User-based access control
- **Secure Login:** Username and password authentication
- **User Registration:** Self-service user registration capability

10.1.3 Major Endpoints

User Management

Endpoint	Method	Description
/api/user/account/info/	GET	Get current user information
/api/user/account/token/	POST	User login and token acquisition
/api/user/account/register/	POST	User registration

Table 10.1: User Management API Endpoints

Device Management

Endpoint	Method	Description
/api/devices/count	GET	Get user's device count
/api/devices	POST	Get user's device list with search
/api/devices/status-summary	GET	Get device status distribution
/api/devices/defective	GET	Get all defective devices
/api/devices/get-device	GET	Get device basic information by ID

Table 10.2: Device Management API Endpoints

Alert Management

Endpoint	Method	Description
/api/alerts/getUnconfirmedAlerts	POST	Get unconfirmed alerts with filters
/api/alerts/confirmAlert	PUT	Batch confirm alert status
/api/alerts/deleteAlerts	DELETE	Batch delete alerts
/api/alerts/exportAlertsToXLSX	GET	Export alerts to Excel file
/api/alerts/status-summary	GET	Get alert status distribution

Table 10.3: Alert Management API Endpoints

Device Monitoring

Endpoint	Method	Description
/api/monitor/{deviceId}/health	GET	Get device health data
/api/monitor/{deviceId}/energy	GET	Get device energy consumption data
/api/monitor/{deviceId}/cards	GET	Get device metric cards data

Table 10.4: Device Monitoring API Endpoints

Report Management

Endpoint	Method	Description
/api/reports/alerts/getTodayAlertStats	GET	Get today's alert statistics
/api/reports/alerts/getAllAlertStats	GET	Get all-time alert statistics
/api/reports/alerts/getWeeklyAlertStats	GET	Get weekly alert statistics
/api/reports/devices/getDeviceAttributes	GET	Get device attributes
/api/reports/devices/exportDeviceAttributes	GET	Export device attributes to Excel

Table 10.5: Report Management API Endpoints

Model Management and Simulation

Endpoint	Method	Description
/api/model/getModels	GET	Get available models list
/api/simulation/getSimulationResult	POST	Get simulation results

Table 10.6: Model Management and Simulation API Endpoints

2 External Interfaces

The system integrates with external systems through the following interfaces:

10.2.1 Data Import and Export

- **File Upload:** Support for CSV, Excel, and binary file uploads
- **Excel Export:** XLSX format export for alerts and device attributes
- **Image Processing:** Support for image uploads in simulation modules
- **Data Validation:** Input validation for uploaded files

10.2.2 Device Integration

- **Health Monitoring:** Real-time device health data acquisition
- **Energy Monitoring:** Energy consumption tracking and cost calculation
- **Status Tracking:** Multi-level device status monitoring (normal, warning, critical)
- **Attribute Management:** Dynamic device attribute configuration and monitoring

10.2.3 Alert and Notification System

- **Real-time Alerts:** Automatic alert generation based on device status
- **Alert Categorization:** Multiple alert status levels with descriptions
- **Batch Operations:** Support for bulk alert confirmation and deletion
- **Time-based Filtering:** Alert filtering by date ranges and device names

3 User Interface to Backend Integration

The integration between frontend and backend components follows these patterns:

- **Data Fetching:**
 - RESTful API calls with JSON payload for complex queries
 - GET requests for simple data retrieval
 - POST requests for search and filtering operations
- **Authentication Flow:**
 - Form-data submission for login credentials
 - JWT token received and stored for session management
 - Token-based authentication for protected endpoints
 - User registration with username, password, and phone validation
- **Error Handling:**
 - Consistent success/error response format with status indicators
 - Detailed error messages for validation failures
 - Graceful handling of batch operation partial failures

- Failed operation tracking with specific ID reporting

- **Data Management:**

- Pagination support for large datasets
- Search and filtering capabilities across multiple endpoints
- Real-time data updates for monitoring dashboards
- File download support for Excel exports

- **Model Integration:**

- Machine learning model selection and execution
- Simulation result processing with visual feedback
- Health index calculation and lifespan prediction
- Damage location identification through image analysis

Chapter11 Non-Functional Design Aspects

1 Performance Design

Performance considerations are crucial for the real-time monitoring and analysis capabilities of the system:

11.1.1 Response Time Goals

- **UI Rendering:** < 2 seconds for initial page load
- **Data Queries:** < 1 second for standard queries
- **Real-time Updates:** < 500ms for dashboard updates
- **Search Operations:** < 3 seconds for complex searches
- **Report Generation:** < 30 seconds for standard reports
- **ML Predictions:** < 5 seconds for RUL prediction

11.1.2 Throughput Requirements

- **Sensor Data Ingestion:** Up to 1000 readings/second per device
- **Concurrent Users:** Support for 1000+ concurrent users
- **API Transactions:** 100+ transactions/second
- **Report Generation:** 50+ reports/hour during peak times

11.1.3 Optimization Strategies

- **Database:**
 - Indexed queries for frequent operations
 - Partitioning for time-series data
 - Read replicas for heavy read workloads
- **Caching:**
 - Multi-level caching (memory, distributed cache)
 - Cache invalidation strategies
 - Time-to-live policies based on data volatility
- **Data Processing:**
 - Parallel processing for batch operations
 - Data downsampling for historical trends
 - Pre-aggregation of common metrics
- **Frontend:**
 - Code splitting and lazy loading
 - Resource compression and minification
 - Progressive loading of content

2 Security Design

Security is paramount given the sensitive nature of industrial equipment monitoring:

11.2.1 Authentication and Authorization

- **Multi-factor Authentication:** Optional two-factor authentication for users
- **Role-Based Access Control:** Granular permissions based on user roles
- **Single Sign-On:** Support for enterprise SSO integration (SAML, OAuth)
- **Password Policies:** Strong password requirements with regular rotation
- **Session Management:** Secure session handling with automatic timeout

11.2.2 Data Protection

- **Encryption at Rest:** Database encryption for sensitive data
- **Encryption in Transit:** TLS 1.3 for all communications
- **Data Masking:** Masking of sensitive information in logs and reports
- **Access Controls:** Principle of least privilege for data access
- **Retention Policies:** Automated enforcement of data retention rules

11.2.3 API Security

- **Rate Limiting:** Prevent abuse through request throttling
- **Input Validation:** Comprehensive validation of all API inputs
- **CORS Policies:** Strict cross-origin resource sharing policies
- **API Keys:** Managed API keys for external integrations
- **Audit Logging:** Detailed logs of all API access and usage

11.2.4 Security Monitoring

- **Intrusion Detection:** Monitoring for suspicious activities
- **Vulnerability Scanning:** Regular automated security scans
- **Audit Logging:** Comprehensive logging of security-relevant events
- **Incident Response:** Defined procedures for security incidents

3 Scalability Design

The system is designed to scale as the number of monitored devices and users increases:

11.3.1 Horizontal Scaling

- **Stateless Services:** Application components designed for horizontal scaling
- **Load Balancing:** Distribution of requests across multiple instances
- **Database Sharding:** Partitioning data across multiple database instances
- **Microservices Architecture:** Independent scaling of system components

11.3.2 Vertical Scaling

- **Resource Allocation:** Ability to increase CPU, memory, and storage
- **Database Optimization:** Index optimization and query tuning
- **Hardware Upgrades:** Support for migrating to more powerful hardware

11.3.3 Elastic Capacity

- **Auto-Scaling:** Automatic adjustment of resources based on load
- **Scheduled Scaling:** Predictive scaling for known usage patterns
- **Burst Handling:** Ability to handle temporary spikes in demand

4 Availability and Reliability Design

The system aims to provide high availability for critical monitoring functions:

11.4.1 Availability Targets

- **Overall System Availability:** 99.9% uptime (8.76 hours downtime per year)
- **Critical Monitoring Functions:** 99.99% uptime (52.6 minutes downtime per year)
- **Scheduled Maintenance:** Less than 4 hours per month during off-peak hours

11.4.2 Reliability Features

- **Redundancy:** Multiple instances of critical components
- **Failover Mechanisms:** Automatic switching to backup systems
- **Circuit Breakers:** Prevention of cascading failures
- **Retry Policies:** Intelligent retry mechanisms for transient errors
- **Data Durability:** Multiple data backups with geographic distribution

11.4.3 Monitoring and Recovery

- **Health Checks:** Regular automated testing of system components
- **Alerting:** Immediate notification of system issues
- **Self-Healing:** Automatic recovery procedures where possible
- **Disaster Recovery:** Comprehensive plan for major outages

Chapter12 Appendices

1 Glossary

Term	Definition
RUL	Remaining Useful Life - The estimated time remaining before a component or system requires maintenance or replacement
PdM	Predictive Maintenance - An approach to maintenance that uses data analysis tools and techniques to detect anomalies and possible defects before they occur
CNN	Convolutional Neural Network - A deep learning algorithm particularly effective for analyzing visual imagery and processing structured grid data
LSTM	Long Short-Term Memory - A type of recurrent neural network capable of learning order dependence in sequence prediction problems
GAN	Generative Adversarial Network - A class of machine learning frameworks where two neural networks contest with each other in a game
DBN	Deep Belief Network - A generative graphical model composed of multiple layers of latent variables with connections between layers but not between units within each layer
BiGRU	Bidirectional Gated Recurrent Unit - A type of RNN that processes sequences in both forward and backward directions
PHOT	Phase-Only Transform - A signal processing technique that preserves phase information while normalizing magnitude information
OEE	Overall Equipment Effectiveness - A measure of manufacturing productivity calculated as the product of Availability, Performance, and Quality

Table 12.1: Glossary of Terms

2 Design Decisions Log

Decision	Rationale	Date	Decision Maker
Adopt multi-layered architecture	To separate concerns and enable modular development	April 3, 2025	Fubin Chen
Use Skip-GANomaly for anomaly detection	Superior performance compared to traditional methods	April 4, 2025	Xuanhe Yang

Continued on next page

Table 12.2 – continued from previous page

Decision	Rationale	Date	Decision Maker
Select CNN-LSTM, CNN-Transformer, and DBN-BiGRU for evaluation	Each offers unique advantages for time series prediction	April 6, 2025	Xuanhe Yang
Implement hybrid database approach	Optimize for both transactional and time-series data	April 10, 2025	Tong Li
Use JWT for authentication	Stateless authentication suitable for distributed systems	April 15, 2025	Jingxiao Han
Adopt containerized deployment	Ensure consistency across environments and enable scaling	April 25, 2025	Project Team
Implement Web-Socket for real-time updates	Reduce latency for dashboard updates	May 2, 2025	Fubin Chen

Table 12.2: Design Decisions Log

3 References

1. Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.
2. Project Management Institute. (2021). A Guide to the Project Management Body of Knowledge (PMBOK Guide) (7th ed.).
3. Sullivan, W. G., Wicks, E. M., & Koelling, C. P. (2023). Engineering Economy (18th ed.). Pearson.
4. Evans, E. (2017). Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley.
5. Martin, R. C. (2018). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.
6. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.